

ĐẠI HỌC BÁCH KHOA HÀ NỘI

Khoa Toán - Tin



Báo cáo Bài tập lớn

**CẤU TRÚC DỮ LIỆU & GIẢI THUẬT
(MI3090)**

Đề tài:

**Xây dựng Hệ thống gợi ý sản phẩm
(Recommendation System)**

Giảng viên hướng dẫn: ThS. Lê Quang Hòa

Mã lớp học: 169308

Nhóm sinh viên thực hiện:

1	Đoàn Đức Anh	202419033	Hệ thống thông tin quản lý 02
2	Ngô Thị Thùy Duyên	202419051	Hệ thống thông tin quản lý 02
3	Nguyễn Đức Thành	202419099	Hệ thống thông tin quản lý 02
4	Phan Thị Hải Yến	202419124	Hệ thống thông tin quản lý 01

Hà Nội, Tháng 6 2026

Lời mở đầu

Trong kỷ nguyên số hóa hiện nay, nền kinh tế thương mại điện tử (E-commerce) đang chứng kiến sự phát triển bùng nổ mạnh mẽ chưa từng có. Cùng với đó là khối lượng thông tin và sản phẩm khổng lồ được cập nhật mỗi ngày. Sự phong phú này mang lại vô vàn lựa chọn cho người tiêu dùng, nhưng đồng thời cũng đặt ra một thách thức lớn: hiện tượng “quá tải thông tin”. Đối với các doanh nghiệp, việc làm thế nào để cá nhân hóa trải nghiệm người dùng, giúp khách hàng nhanh chóng tìm thấy đúng sản phẩm họ cần giữa hàng triệu lựa chọn, đã trở thành chìa khóa cốt lõi để duy trì tính cạnh tranh. Nhận thức được bức thiết đó, Hệ thống gợi ý (Recommendation System) ra đời như một giải pháp tất yếu. Đóng vai trò là một “trợ lý ảo” đắc lực, hệ thống này giúp kết nối sản phẩm với đúng nhu cầu của khách hàng thông qua việc phân tích và khai thác lịch sử tương tác.

Trong khuôn khổ môn học Cấu trúc Dữ liệu và Giải thuật (DSA), thay vì phụ thuộc vào các thư viện hay framework có sẵn, việc đi sâu vào bản chất và tự tay xây dựng nền móng cho một hệ thống gợi ý là một trải nghiệm mang tính thử thách cao và vô cùng giá trị. Chính vì lý do đó, dự án “Xây dựng hệ thống gợi ý sản phẩm dựa trên lịch sử tương tác” đã được lựa chọn và thực hiện. Điểm nhấn đặc biệt của dự án này nằm ở khía cạnh kỹ thuật: cốt lõi của hệ thống (Core Engine) được thiết kế và cài đặt hoàn toàn bằng ngôn ngữ C++. Dự án tập trung ứng dụng linh hoạt các nền tảng cấu trúc dữ liệu như Mảng động (Dynamic Array), Bảng băm (Hash Table) và các thuật toán cơ bản (như Bubble Sort,...). Mục tiêu lớn nhất là tối ưu hóa hiệu năng, đảm bảo tốc độ phản hồi nhanh chóng và hiệu quả về mặt bộ nhớ.

Thông qua dự án này, chúng em không chỉ mong muốn giải quyết một bài toán thực tiễn hấp dẫn, mà còn kỳ vọng củng cố vững chắc tư duy thuật toán – thứ vũ khí quan trọng nhất của mọi kỹ sư phần mềm. Báo cáo dưới đây sẽ trình bày chi tiết tiến trình thực hiện dự án: từ tổng quan bài toán, cơ sở lý thuyết, thiết kế hệ thống, cho đến quá trình cài đặt và đánh giá kết quả thực tế.

Nhân dịp này, chúng em xin gửi lời cảm ơn tới thầy Lê Quang Hòa, với những kiến thức, bài giảng và góp ý nhận xét giúp chúng em có thể hoàn thành dự án này. Mặc dù đã nỗ lực hết sức trong quá trình nghiên cứu và phát triển, song dự án chắc chắn không tránh khỏi những hạn chế và thiếu sót nhất định. Chúng em rất mong nhận được những nhận xét, góp ý quý báu từ quý Thầy để có thể tiếp tục hoàn thiện và phát triển hệ thống trong tương lai.

Nhóm chân thành cảm ơn!

Danh sách hình vẽ

5.1	Kết quả chạy demo console - luồng Auth, Item và Cart/Order	28
5.2	Giao diện ReactJS hiển thị danh sách gợi ý sản phẩm cá nhân hóa	28
5.3	Kết quả kiểm thử TC01–TC03: Nạp dữ liệu từ CSV	29
5.4	Kết quả kiểm thử TC13–TC16: Gợi ý sản phẩm cá nhân hóa	31
5.5	Kết quả kiểm thử TC13–TC16: Gợi ý sản phẩm cá nhân hóa	31
5.6	Kết quả kiểm thử TC19–TC23: Luồng xác thực người dùng	32
5.7	Kết quả kiểm thử TC19–TC23: Luồng xác thực người dùng	32
5.8	Kết quả kiểm thử TC24–TC27: Quản lý sản phẩm	33
5.9	Kết quả kiểm thử TC24–TC27: Quản lý sản phẩm	33
5.10	Kết quả kiểm thử TC24–TC27: Quản lý sản phẩm	34
5.11	Kết quả kiểm thử TC28–TC31: Quản lý giỏ hàng	34
5.12	Kết quả kiểm thử TC32–TC37: Đặt hàng, gợi ý chéo và kiểm thử API	35

Danh sách bảng

1.1	Bảng công nghệ sử dụng trong dự án	8
3.1	Kiến trúc bốn tầng của hệ thống	13
3.2	Bảng Use Case tổng hợp của hệ thống	14
3.3	Thiết kế các lớp thực thể (1_models)	15
3.4	Bảng trọng số hành vi trong hàm <code>computeScore()</code>	16
3.5	Quan hệ giữa các thực thể trong hệ thống	17
3.6	Cấu trúc các file CSV dữ liệu	17
3.7	Lược đồ MySQL của hệ thống	18
3.8	Bảng thiết kế REST API Endpoints	18
4.1	Các thao tác của MyVector và độ phức tạp	22
4.2	Phân tích độ phức tạp thời gian của các thao tác trên Hash Map	23
4.3	Phân tích độ phức tạp thời gian của Bubble Sort	24
4.4	Phân tích độ phức tạp không gian của Bubble Sort	25
4.5	So sánh độ phức tạp tính toán Cosine Similarity	26
5.1	Môi trường thực nghiệm	27
5.2	Kiểm thử Nhóm 1: Nạp và xử lý dữ liệu	29
5.3	Kiểm thử Nhóm 2: Hàm tính điểm <code>computeScore()</code>	29
5.4	Kiểm thử Nhóm 3: Xây dựng Rating Matrix	30
5.5	Kiểm thử Nhóm 4: Tính Similarity Matrix	30
5.6	Kiểm thử Nhóm 5: Engine gợi ý cá nhân hóa	30
5.7	Kiểm thử Nhóm 6: Gợi ý Cold Start	31
5.8	Kiểm thử Nhóm 7: Xác thực người dùng	32
5.9	Kiểm thử Nhóm 8: Quản lý sản phẩm	33
5.10	Kiểm thử Nhóm 9: Quản lý giỏ hàng	34
5.11	Kiểm thử Nhóm 10: Đặt hàng, gợi ý chéo và API	35
5.12	Bảng tổng hợp kết quả kiểm thử 37 test case	36
6.1	Tổng kết các kỹ thuật lập trình đã vận dụng	38
6.2	Các hạn chế hiện tại của hệ thống	38
3	Yêu cầu hệ thống	41
4	Danh sách các API Endpoints	44

Mục lục

Lời mở đầu	1
Danh sách hình vẽ	2
Danh sách bảng	3
1 Giới thiệu bài toán	6
1.1 Bối cảnh nghiên cứu	6
1.2 Vấn đề nghiên cứu	6
1.3 Mục tiêu của dự án	7
1.4 Phạm vi và giới hạn thực hiện	7
1.5 Công nghệ và ngôn ngữ lập trình sử dụng	8
2 Cơ sở lý thuyết	9
2.1 Lý thuyết về Hệ thống gợi ý (Recommendation System)	9
2.1.1 Thuật toán Lọc cộng tác dựa trên sản phẩm (Item-based Collaborative Filtering)	9
2.1.2 Công thức Toán học: Tính độ tương đồng Cosine (Cosine Similarity)	9
2.1.3 Vấn đề người dùng mới và giải pháp Cold Start	10
2.2 Cấu trúc dữ liệu	10
2.2.1 Mảng động (Dynamic Array)	10
2.2.2 Bảng băm (Hash Table)	11
2.2.3 Thuật toán Sắp xếp nổi bọt (Bubble Sort)	11
2.3 Kiểm thử phần mềm	12
2.3.1 Kiểm thử hộp đen (Black-Box Testing)	12
2.3.2 Kiểm thử hộp trắng (White-Box Testing)	12
2.3.3 Kiểm thử hộp xám (Gray-Box Testing)	12
2.3.4 Kiểm thử tự động (Automation Test)	12
3 Phân tích và Thiết kế hệ thống	13
3.1 Kiến trúc tổng thể	13
3.2 Phân tích yêu cầu và Use Case	13
3.2.1 Danh sách các tác nhân (Actors)	13
3.2.2 Bảng Use Case tổng hợp	14
3.2.3 Luồng Use Case chính: Gợi ý sản phẩm cá nhân hóa (UC-08)	14
3.3 Luồng hoạt động của hàm main()	15
3.4 Thiết kế lớp (Class Design)	15
3.4.1 Tầng Thực thể (1_models)	15
3.4.2 Tầng Hạ tầng (2_data)	16
3.4.3 Tầng Thuật toán (3_core)	16
3.4.4 Tầng Nghiệp vụ (6_services)	16
3.5 Thiết kế quan hệ dữ liệu	17
3.5.1 Quan hệ giữa các thực thể	17
3.5.2 Thiết kế cơ sở dữ liệu CSV	17
3.5.3 Lược đồ MySQL	18
3.6 Thiết kế API Endpoints	18
4 Cài đặt hệ thống	19
4.1 Cấu trúc thư mục chương trình thực tế	19
4.2 Triển khai xây dựng các hàm chính	20

4.2.1	Cấu trúc cặp giá trị <code>MyPair</code> (Cài đặt thay thế <code>std::pair</code>)	20
4.2.2	Cấu trúc <code>MyVector</code> (Cài đặt thay thế <code>std::vector</code>)	21
4.2.3	Bảng băm <code>MyMap</code> (Cài đặt thay thế <code>std::unordered_map</code>)	22
4.3	Các thuật toán nền tảng	24
4.3.1	Thuật toán Sắp xếp nổi bọt (Bubble Sort)	24
4.3.2	Triển khai hàm tính toán Cosine Similarity tối ưu bằng kỹ thuật <code>smallerMap</code> . . .	25
5	Thực nghiệm và đánh giá kết quả	27
5.1	Môi trường thực nghiệm	27
5.2	Kết quả chạy demo console	27
5.3	Kết quả gợi ý sản phẩm	28
5.4	Kiểm thử các chức năng theo luồng nghiệp vụ	28
5.4.1	Nhóm 1: Nạp và xử lý dữ liệu (TC01–TC03)	29
5.4.2	Nhóm 2: Tính điểm tương tác - <code>computeScore()</code> (TC04–TC06)	29
5.4.3	Nhóm 3: Xây dựng Rating Matrix (TC07–TC09)	30
5.4.4	Nhóm 4: Tính Similarity Matrix (TC10–TC12)	30
5.4.5	Nhóm 5: Engine gợi ý cá nhân hóa - <code>forUser()</code> (TC13–TC16)	30
5.4.6	Nhóm 6: Gợi ý Cold Start (TC17–TC18)	31
5.4.7	Nhóm 7: Xác thực người dùng - Auth (TC19–TC23)	32
5.4.8	Nhóm 8: Quản lý sản phẩm (TC24–TC27)	33
5.4.9	Nhóm 9: Quản lý giỏ hàng (TC28–TC31)	34
5.4.10	Nhóm 10: Đặt hàng và gợi ý chéo (TC32–TC37)	35
5.5	Tổng hợp kết quả kiểm thử	36
6	Kết luận và Hướng phát triển	37
6.1	Kết quả đạt được	37
6.2	Tổng kết các kỹ thuật lập trình đã vận dụng	38
6.3	Hạn chế của hệ thống	38
6.4	Hướng phát triển và nâng cấp tương lai	39
	Tài liệu tham khảo	39
	Phụ lục A: Hướng dẫn triển khai hệ thống	41
	Phụ lục B: Mã nguồn các thuật toán cốt lõi	45

Chương 1

Giới thiệu bài toán

1.1 Bối cảnh nghiên cứu

Sự phát triển mạnh mẽ của Internet và các nền tảng số đã thúc đẩy thương mại điện tử trở thành một trong những lĩnh vực tăng trưởng nhanh nhất của nền kinh tế toàn cầu. Theo báo cáo của Forrester Research, doanh thu bán lẻ thương mại điện tử toàn cầu tăng từ 1,3 nghìn tỷ USD năm 2014 lên 4,4 nghìn tỷ USD năm 2023 và dự kiến đạt 6,8 nghìn tỷ USD vào năm 2028 [1]. Tốc độ tăng trưởng kép hàng năm (CAGR) đạt trên 9% trong giai đoạn này, vượt xa phần lớn các ngành công nghiệp truyền thống. Tuy nhiên, sự tăng trưởng vượt bậc về quy mô kéo theo một hệ quả tất yếu: số lượng sản phẩm được cung cấp trên các nền tảng trực tuyến ngày càng khổng lồ. Một sàn thương mại điện tử lớn như Amazon hiện có hơn 350 triệu sản phẩm, Shopee Đông Nam Á có hơn 200 triệu sản phẩm đang hoạt động.

Sự bùng nổ về lựa chọn này đẩy người tiêu dùng vào trạng thái **quá tải thông tin (Information Overload)**: nghiên cứu của Gartner chỉ ra rằng 38% khách hàng đã từ bỏ một phiên mua sắm vì bị áp đảo bởi quá nhiều lựa chọn không liên quan [2]. Trong bối cảnh đó, **cá nhân hóa trải nghiệm người dùng** đã trở thành yêu cầu chiến lược, không còn là điểm khác biệt tùy chọn. Báo cáo của McKinsey & Company cho thấy 71% khách hàng kỳ vọng nhận được trải nghiệm cá nhân hóa, và 76% trong số họ cảm thấy thất vọng khi doanh nghiệp không đáp ứng được nhu cầu này [3]. Đáng chú ý hơn, nghiên cứu của Salesforce Research chỉ ra rằng 65% khách hàng có xu hướng trung thành hơn với thương hiệu khi được cung cấp trải nghiệm cá nhân hóa [4].

Về phía doanh nghiệp, tác động kinh tế của cá nhân hóa thông qua hệ thống gợi ý là hoàn toàn có thể đo lường được. McKinsey ước tính rằng cơ chế gợi ý sản phẩm đóng góp khoảng 35% tổng doanh thu của Amazon, trong khi tại Netflix, hơn 75% nội dung người dùng xem đều xuất phát từ các đề xuất cá nhân hóa [3]. Đồng thời, khảo sát của Salesforce cũng cho thấy 52% khách hàng kỳ vọng các ưu đãi và đề xuất từ thương hiệu phải được cá nhân hóa theo đúng sở thích của họ - nếu hệ thống gợi ý sai nhu cầu, tỷ lệ thoát trang (bounce rate) tăng cao, trực tiếp ảnh hưởng đến doanh thu [4].

1.2 Vấn đề nghiên cứu

Dựa trên bối cảnh trên, bài toán được đặt ra có thể phân tách thành hai thách thức cốt lõi:

Thách thức 1 - Quá tải thông tin (Information Overload): Khi danh mục sản phẩm lên đến hàng trăm triệu mặt hàng, chiến lược hiển thị “phổ thông” (tất cả người dùng nhìn thấy cùng một trang chủ) trở nên hoàn toàn không hiệu quả. Người dùng mất quá nhiều thời gian để tìm kiếm, dẫn đến hành vi bỏ cuộc (cart abandonment), ảnh hưởng tiêu cực đến tỷ lệ chuyển đổi (conversion rate). Bài toán đặt ra là: làm thế nào để từ hàng triệu sản phẩm, hệ thống lọc và đưa ra danh sách chỉ vài chục mặt hàng *thực sự phù hợp* với từng người dùng cụ thể?

Thách thức 2 - Khai thác dữ liệu hành vi ngầm (Implicit Feedback): Trong thực tế triển khai, chỉ một phần nhỏ người dùng chủ động để lại đánh giá (rating) bằng số sao. Phần lớn dữ liệu hành vi tồn tại dưới dạng *tín hiệu ngầm*: click xem sản phẩm, thêm vào giỏ hàng, hoàn tất giao dịch. Bài toán đặt ra là: làm thế nào để xây dựng một hàm tính điểm (scoring function) kết hợp nhiều loại tín hiệu hành vi với trọng số phản ánh đúng mức độ quan tâm thực sự của người dùng?

Dự án này giải quyết đồng thời cả hai thách thức trên thông qua việc xây dựng một **Hệ thống gợi ý sản phẩm (Product Recommendation System)** áp dụng phương pháp Lọc cộng tác dựa trên sản

phẩm (Item-based Collaborative Filtering) kết hợp đo lường độ tương đồng Cosine (Cosine Similarity), với toàn bộ lỗi xử lý được cài đặt bằng C++ thuần không phụ thuộc STL.

1.3 Mục tiêu của dự án

Dự án đặt ra các mục tiêu cụ thể sau:

1. **Xây dựng Engine gợi ý cốt lõi:** Cài đặt thuật toán Item-based Collaborative Filtering kết hợp Cosine Similarity hoàn toàn bằng C++ thuần, không sử dụng STL container hay thư viện ML bên ngoài.
2. **Tự triển khai các cấu trúc dữ liệu nền tảng:** Xây dựng **MyVector** (mảng động), **MyMap** (bảng băm), **MyPair** (cặp khóa-giá trị) từ đầu bằng con trỏ và quản lý bộ nhớ thủ công.
3. **Thiết kế hàm tính điểm đa hành vi:** Xây dựng hàm **computeScore()** kết hợp bốn tín hiệu hành vi (click, add_cart, purchase, rating) với trọng số phản ánh mức độ cam kết thực sự của người dùng.
4. **Giải quyết bài toán Cold Start:** Xây dựng cơ chế gợi ý dự phòng (fallback) dựa trên độ phổ biến tổng hợp cho người dùng mới chưa có lịch sử tương tác.
5. **Xây dựng REST API:** Tích hợp Crow/ASIO để cung cấp các endpoint API phục vụ giao diện Frontend (ReactJS + Vite).
6. **Tích hợp đa nguồn dữ liệu:** Hỗ trợ đọc/ghi CSV và đồng bộ với MySQL qua **DatabaseManager**.

1.4 Phạm vi và giới hạn thực hiện

Phạm vi trong phạm vi thực hiện:

- Xây dựng engine gợi ý dựa trên Item-based Collaborative Filtering với Cosine Similarity, không phụ thuộc vào thư viện ML bên ngoài.
- Tự cài đặt các cấu trúc dữ liệu nền tảng (**MyVector**, **MyMap**, **MyPair**) không dùng STL containers.
- Xây dựng đầy đủ luồng nghiệp vụ: đăng ký/dăng nhập, tìm kiếm sản phẩm, giỏ hàng, đặt hàng và gợi ý chéo.
- Triển khai API server tại **localhost:8080** để kết nối với giao diện ReactJS.
- Kiểm thử trên tập dữ liệu mẫu: 10.000 người dùng, 500 sản phẩm, 100.000 tương tác.

Giới hạn không nằm trong phạm vi:

- Hệ thống chưa triển khai lên môi trường Internet hoặc cloud; mọi thành phần chạy trên localhost.
- Chưa xử lý đồng thời (concurrency) cho lượng truy cập lớn; Crow server đơn giản được dùng để minh họa kiến trúc API.
- Mật khẩu người dùng chưa được băm (hashing); đây là giới hạn có chủ ý trong phạm vi học thuật.
- Chưa xử lý Cold Start cho *sản phẩm mới* (New Item Cold Start), chỉ xử lý New User Cold Start.

1.5 Công nghệ và ngôn ngữ lập trình sử dụng

Thành phần	Công nghệ	Vai trò
Ngôn ngữ lập trình chính	C++17	Xây dựng Recommendation Engine
Lập trình hướng đối tượng	OOP C++	Tổ chức các module hệ thống
Cấu trúc dữ liệu	MyVector, MyMap, MyPair	Quản lý dữ liệu trong bộ nhớ
Thuật toán gợi ý	Item-based CF	Sinh danh sách sản phẩm đề xuất
Độ đo tương đồng	Cosine Similarity	Tính mức độ giống nhau giữa các sản phẩm
API Framework	Crow C++	Cung cấp REST API
Frontend	ReactJS + Vite	Xây dựng giao diện người dùng
Định dạng dữ liệu	JSON	Trao đổi dữ liệu giữa Client và Server
Lưu trữ dữ liệu	CSV Files + MySQL	Lưu dữ liệu hệ thống
Công cụ phát triển	Visual Studio Code	Lập trình và kiểm thử
Quản lý phiên bản	Git/GitHub	Quản lý mã nguồn

Bảng 1.1: Bảng công nghệ sử dụng trong dự án

Chương 2

Cơ sở lý thuyết

2.1 Lý thuyết về Hệ thống gợi ý (Recommendation System)

Hệ thống gợi ý (Recommendation System) là một phân lớp của hệ thống lọc thông tin, được thiết kế nhằm dự đoán “độ ưu tiên” hoặc “đánh giá” mà một người dùng sẽ dành cho một vật phẩm (item) [5]. Trong kỷ nguyên bùng nổ dữ liệu số, hệ thống gợi ý đóng vai trò thiết yếu trong việc giải quyết vấn đề quá tải thông tin, giúp cá nhân hóa trải nghiệm người dùng trên các nền tảng thương mại điện tử và dịch vụ nội dung.

2.1.1 Thuật toán Lọc cộng tác dựa trên sản phẩm (Item-based Collaborative Filtering)

Lọc cộng tác (Collaborative Filtering - CF) là một trong những phương pháp nền tảng và thành công nhất trong lĩnh vực hệ thống gợi ý. Phương pháp này dựa trên giả định cốt lõi: những người dùng từng có sở thích giống nhau trong quá khứ sẽ có xu hướng đánh giá cao những sản phẩm giống nhau trong tương lai [5]. Thay vì tìm kiếm sự tương đồng giữa các người dùng (User-based CF) vốn tốn nhiều tài nguyên tính toán do số lượng người dùng thường xuyên biến động, hệ thống của dự án áp dụng phương pháp Lọc cộng tác dựa trên sản phẩm (Item-based Collaborative Filtering). Thuật toán này được giới thiệu lần đầu bởi nhóm nghiên cứu tại Đại học Minnesota (2001) và ứng dụng thành công tại Amazon [6].

Logic hoạt động của Item-based CF bao gồm các bước sau:

- Phân tích tập dữ liệu:** Xây dựng ma trận tương tác User-Item (Người dùng - Sản phẩm), trong đó mỗi ô lưu giá trị đánh giá (rating) hoặc mức độ tương tác (click, thêm vào giỏ hàng, mua hàng).
- Tính toán độ tương đồng:** Đánh giá mức độ giống nhau giữa các cặp sản phẩm dựa trên lịch sử tương tác của tất cả người dùng đối với hai sản phẩm đó.
- Dự đoán và Gợi ý:** Nếu người dùng u đã tương tác tích cực với tập sản phẩm S , hệ thống sẽ tìm kiếm các sản phẩm chưa được tương tác có độ tương đồng cao nhất với các sản phẩm trong tập S để đưa ra danh sách gợi ý.

Ưu điểm nổi bật của Item-based CF là tính ổn định. Tập hợp sản phẩm thường có tính tĩnh cao hơn tập hợp người dùng, do đó ma trận tương đồng sản phẩm có thể được tính toán ngoại tuyến (offline), giúp tăng tốc độ phản hồi của hệ thống theo thời gian thực [7].

2.1.2 Công thức Toán học: Tính độ tương đồng Cosine (Cosine Similarity)

Để đo lường mức độ tương đồng giữa hai sản phẩm trong ma trận tương tác, thuật toán sử dụng phép đo Độ tương đồng Cosine (Cosine Similarity). Khái niệm này coi mỗi sản phẩm như một vector đa chiều trong không gian, nơi mỗi chiều biểu diễn mức độ tương tác của một người dùng cụ thể [8]. Độ tương đồng giữa hai sản phẩm i và j được tính bằng cosin của góc tạo bởi hai vector tương ứng của chúng. Góc càng nhỏ, giá trị cosin càng tiến về 1, thể hiện hai sản phẩm có mức độ đánh giá rất giống nhau. Ngược lại, giá trị tiến về 0 thể hiện sự thiếu tương đồng.

Công thức tổng quát tính độ tương đồng Cosine giữa hai vector sản phẩm $\vec{i}$ và $\vec{j}$ được biểu diễn như sau:

$$sim(i, j) = \cos(\vec{i}, \vec{j}) = \frac{\vec{i} \cdot \vec{j}}{|\vec{i}| \times |\vec{j}|}$$

Khi triển khai trên ma trận tương tác, công thức trên được khai triển thành:

$$\text{sim}(i, j) = \frac{\sum_{u \in U_{i,j}} R_{u,i} \times R_{u,j}}{\sqrt{\sum_{u \in U_i} R_{u,i}^2} \times \sqrt{\sum_{u \in U_j} R_{u,j}^2}}$$

Trong đó:

- $\text{sim}(i, j)$: Mức độ tương đồng giữa sản phẩm i và sản phẩm j .
- $R_{u,i}, R_{u,j}$: Điểm đánh giá (hoặc trọng số tương tác) của người dùng u đối với sản phẩm i và j .
- $U_{i,j}$: Tập hợp những người dùng đã tương tác với cả hai sản phẩm i và j .
- U_i, U_j : Tập hợp những người dùng đã tương tác với sản phẩm i và sản phẩm j .

Lưu ý triển khai: Để tối ưu hóa tài nguyên tính toán (như đã thiết kế trong kiến trúc C++ của hệ thống), tử số chỉ thực hiện tính tổng tích vô hướng đối với tập giao $U_{i,j}$ của hai sản phẩm, loại bỏ các phép tính thừa bằng 0.

2.1.3 Vấn đề người dùng mới và giải pháp Cold Start

Cold Start là một trong những thách thức kinh điển và lớn nhất đối với mọi hệ thống lọc cộng tác [9]. Vấn đề này xảy ra khi hệ thống không có đủ dữ liệu lịch sử tương tác để đưa ra các suy luận và gợi ý chính xác. Trong khuôn khổ đồ án, hệ thống tập trung giải quyết nhánh New User Cold Start (Người dùng mới): Khi một người dùng vừa đăng ký tài khoản và chưa có bất kỳ thao tác click, mua hàng hay đánh giá nào, thuật toán Item-based CF không thể tìm thấy tập sản phẩm S để làm cơ sở tính toán khoảng cách vector. Nếu không được xử lý, hệ thống sẽ trả về danh sách rỗng, gây ảnh hưởng tiêu cực đến trải nghiệm người dùng (User Experience).

Giải pháp tiếp cận của hệ thống: Để khắc phục vấn đề này, cơ chế gợi ý của chương trình áp dụng phương pháp Fallback (Dự phòng) phi cá nhân hóa. Cụ thể, khi định danh người dùng (`user_id`) chưa tồn tại trong ma trận tương tác, hệ thống sẽ tự động chuyển hướng sang hàm xử lý `coldStart`. Tại đây, hệ thống hoạt động như một thuật toán thống kê đại chúng (Popularity-based Recommendation). Chương trình sẽ duyệt qua toàn bộ tập dữ liệu cấu trúc, tổng hợp tổng điểm tương tác (`itemTotalScore`) của từng sản phẩm, sau đó tiến hành sắp xếp giảm dần (Sorting) để trích xuất ra Top N sản phẩm thịnh hành nhất. Danh sách này sẽ được dùng làm gợi ý mặc định ban đầu. Ngay khi người dùng mới thực hiện những thao tác tương tác đầu tiên, hệ thống sẽ lập tức kích hoạt lại ma trận Cosine để chuyển đổi sang luồng gợi ý cá nhân hóa thời gian thực.

2.2 Cấu trúc dữ liệu

Căn cứ theo tiêu chuẩn *C++ Core Guidelines* do Bjarne Stroustrup và Herb Sutter biên soạn, khi tự cài đặt các cấu trúc dữ liệu cấp phát động (như `MyVector`), hệ thống phải tuân thủ nghiêm ngặt “Quy tắc Tam” (The Rule of Three). Cụ thể, nếu một lớp (class) tự quản lý tài nguyên thông qua con trỏ thô, lập trình viên bắt buộc phải tự định nghĩa đồng bộ cả 3 thành phần sau:

- **Hàm hủy (Destructor):** Đảm bảo giải phóng vùng nhớ (dùng `delete[]`) khi đối tượng kết thúc phạm vi hoạt động (out-of-scope).
- **Hàm tạo sao chép (Copy Constructor):** Thực hiện sao chép sâu (deep copy) bằng cách xin cấp phát một vùng nhớ hoàn toàn mới thay vì chỉ copy địa chỉ con trỏ.
- **Toán tử gán (Copy Assignment Operator):** Phải dọn dẹp vùng nhớ cũ của đối tượng đích trước khi cấp phát và copy dữ liệu mới sang.

Việc thiếu hụt bất kỳ thành phần nào trong bộ ba này sẽ khiến C++ tự động thực hiện sao chép nông (shallow copy). Điều này dẫn đến việc nhiều đối tượng cùng trỏ vào chung một vùng nhớ, gây ra lỗi rò rỉ bộ nhớ (memory leak) hoặc lỗi giải phóng kép (*double-free*) làm sập chương trình [10].

2.2.1 Mảng động (Dynamic Array)

Mảng động (Dynamic Array) là một cấu trúc dữ liệu tuyến tính cho phép lưu trữ một tập hợp các phần tử có cùng kiểu dữ liệu tại các vị trí bộ nhớ liên tiếp nhau. Khác với mảng tĩnh (Static Array) yêu cầu khai báo trước kích thước cố định, mảng động sở hữu khả năng tự động điều chỉnh sức chứa (capacity) trong quá trình thực thi (runtime) [11].

Cơ chế hoạt động: Bản chất của mảng động vẫn là một vùng nhớ tĩnh được cấp phát trên bộ nhớ Heap. Theo cấu trúc dữ liệu chuẩn [11], nó quản lý ba thuộc tính cốt lõi:

- **Con trỏ dữ liệu:** Trỏ đến địa chỉ bắt đầu của vùng nhớ được cấp phát.
- **Kích thước (Size):** Số lượng phần tử thực tế đang được lưu trữ.
- **Sức chứa (Capacity):** Tổng số phần tử tối đa mà vùng nhớ hiện tại có thể chứa.

Cơ chế mở rộng (Resizing): Khi thao tác thêm phần tử làm cho kích thước vượt quá sức chứa hiện tại ($\text{Size} = \text{Capacity}$), mảng động sẽ thực hiện chiến lược mở rộng:

- Cấp phát một vùng nhớ mới có kích thước lớn hơn (thường là gấp đôi, $\text{Capacity} \times 2$).
- Sao chép toàn bộ dữ liệu từ vùng nhớ cũ sang vùng nhớ mới.
- Giải phóng vùng nhớ cũ và cập nhật con trỏ đến vùng nhớ mới.

Nhờ chiến lược cấp phát lại theo cấp số nhân này, phân tích khấu hao (Amortized Analysis) chỉ ra rằng độ phức tạp thời gian trung bình (Amortized Time Complexity) cho thao tác thêm một phần tử vào cuối mảng chỉ là $O(1)$, mang lại hiệu năng tối ưu cho việc đọc/ghi dữ liệu tuần tự với số lượng lớn [11].

2.2.2 Bảng băm (Hash Table)

Bảng băm (Hash Table) là cấu trúc dữ liệu chuyên dụng để triển khai kiểu dữ liệu trừu tượng Từ điển (Dictionary) hoặc Bản đồ (Map), cho phép ánh xạ các khóa (Keys) tới các giá trị (Values) tương ứng. Điểm mạnh lớn nhất của Bảng băm là khả năng thực hiện các thao tác tìm kiếm, chèn và xóa với độ phức tạp thời gian trung bình đạt $O(1)$ [12].

Cơ chế hoạt động: Bảng băm sử dụng một hàm toán học gọi là Hàm băm (Hash Function) để biến đổi một khóa đầu vào (ví dụ: chuỗi ký tự định danh `user_id`) thành một số nguyên. Số nguyên này sau đó được chia lấy dư (modulo) cho kích thước của một mảng chứa (Bucket array) để tạo ra chỉ mục (Index) phân bổ vị trí lưu trữ [12].

Xử lý đụng độ (Collision Resolution): Trong thực tế, do không gian của khóa thường lớn hơn rất nhiều so với không gian của mảng chứa, hiện tượng đụng độ (Collision) chắc chắn sẽ xảy ra - tức là hai khóa khác nhau bị hàm băm ánh xạ vào cùng một chỉ mục. Để giải quyết vấn đề này, phương pháp Phân giải bằng danh sách liên kết (Chaining) được sử dụng phổ biến [12]:

- Mỗi bucket trong mảng băm không chứa trực tiếp dữ liệu, mà chứa một con trỏ trỏ tới phần tử đầu tiên của một Danh sách liên kết đơn (Singly Linked List).
- Khi có đụng độ xảy ra tại một chỉ mục, phần tử mới sẽ được chèn thêm vào danh sách liên kết tại bucket đó.
- Khi tìm kiếm, hệ thống tính toán chỉ mục, sau đó duyệt qua danh sách liên kết tại bucket tương ứng để trích xuất khóa chính xác.

Hiệu năng của Bảng băm phụ thuộc rất lớn vào khả năng phân phối đều dữ liệu của Hàm băm (Uniform Hashing) và việc duy trì Tỷ lệ tải (Load Factor) ở mức hợp lý.

2.2.3 Thuật toán Sắp xếp nổi bọt (Bubble Sort)

Sắp xếp nổi bọt (Bubble Sort) là một trong những thuật toán sắp xếp cơ bản và trực quan nhất trong khoa học máy tính. Thuật toán thuộc lớp thuật toán sắp xếp tại chỗ (In-place sorting), không đòi hỏi thêm không gian bộ nhớ phụ trợ đáng kể trong quá trình thực thi [13].

Cơ chế hoạt động: Thuật toán hoạt động dựa trên nguyên lý lặp đi lặp lại việc duyệt qua danh sách cần sắp xếp. Tại mỗi bước duyệt, nó so sánh hai phần tử liên tiếp và tiến hành hoán đổi (swap) vị trí của chúng nếu chúng đang ở sai thứ tự (ví dụ: phần tử trước lớn hơn phần tử sau trong bài toán sắp xếp tăng dần) [13]. Quá trình này được lặp lại cho đến khi không còn bất kỳ sự hoán đổi nào cần thực hiện trong một vòng duyệt toàn bộ danh sách, báo hiệu rằng mảng đã được sắp xếp hoàn chỉnh. Tên gọi “nổi bọt” xuất phát từ việc các phần tử có giá trị lớn nhất (hoặc nhỏ nhất) sẽ dần dần “nổi” lên vị trí cuối cùng của danh sách sau mỗi vòng lặp.

Đánh giá độ phức tạp [13]:

- **Thời gian:** Do sử dụng hai vòng lặp lồng nhau để so sánh mọi cặp phần tử, độ phức tạp thời gian trong trường hợp trung bình và xấu nhất đều là $O(n^2)$.

- **Không gian:** Thuật toán chỉ sử dụng một biến tạm thời để thực hiện thao tác hoán đổi, do đó độ phức tạp không gian đạt $O(1)$.

Mặc dù không mang lại hiệu năng cao trên tập dữ liệu lớn so với các thuật toán tối ưu (như Quick Sort hay Merge Sort), Bubble Sort vẫn mang lại giá trị ứng dụng trong các trường hợp tập dữ liệu có kích thước rất nhỏ hoặc mảng gần như đã được sắp xếp sẵn, nhờ vào sự đơn giản trong cài đặt và không phát sinh chi phí gọi hàm đệ quy.

2.3 Kiểm thử phần mềm

2.3.1 Kiểm thử hộp đen (Black-Box Testing)

Kiểm thử hộp đen là phương pháp kiểm thử trong đó người kiểm thử đóng vai trò là người sử dụng cuối (End-user). Quá trình kiểm thử tập trung vào việc xác minh các chức năng của phần mềm thông qua giao diện người dùng mà không cần biết hay quan tâm đến cấu trúc mã nguồn bên trong.

- **Kiểm chứng giá trị biên (Boundary Testing):** Kỹ thuật thiết kế các trường hợp kiểm thử (Test case) tập trung vào các điều kiện ở các vùng biên của không gian dữ liệu đầu vào. Ví dụ: Kiểm tra phản ứng của hệ thống khi nhập dữ liệu trống, dữ liệu tối đa, hoặc dữ liệu vượt quá giới hạn cho phép.
- **Kiểm chứng tải (Stress Testing):** Kỹ thuật đánh giá sức chịu đựng và độ ổn định của hệ thống khi phải hoạt động dưới mức tải tối đa hoặc vượt quá ngưỡng thiết kế. Ví dụ: Cung cấp khối lượng dữ liệu đầu vào khổng lồ hoặc tạo ra số lượng truy cập đồng thời cực lớn để quan sát hiện tượng tràn bộ nhớ hoặc gián đoạn dịch vụ.

2.3.2 Kiểm thử hộp trắng (White-Box Testing)

Kiểm thử hộp trắng là phương pháp kiểm thử phần mềm dựa trên việc phân tích chi tiết cấu trúc bên trong, mã nguồn, thuật toán và luồng điều khiển của chương trình.

- **Kiểm tra bất biến (Testing Invariants):** Đảm bảo các cấu trúc dữ liệu và các điều kiện logic cốt lõi của hệ thống luôn được bảo toàn và không bị sai lệch trong quá trình thực thi các thao tác.
- **Kiểm tra giá trị trả về (Checking function return values):** Kiểm tra và xác minh các giá trị đầu ra của từng hàm hoặc phương thức cụ thể nhằm đảm bảo việc xử lý ngoại lệ và luồng điều hướng của chương trình diễn ra chính xác theo thiết kế.

2.3.3 Kiểm thử hộp xám (Gray-Box Testing)

Kiểm thử hộp xám là phương pháp lai kết hợp những ưu điểm của cả kiểm thử hộp đen và kiểm thử hộp trắng. Phương pháp này được áp dụng khi người kiểm thử có kiến thức về một phần cấu trúc bên trong, thiết kế hệ thống hoặc cơ sở dữ liệu của phần mềm.

- **Nguyên lý hoạt động:** Người kiểm thử thực hiện các thao tác nghiệp vụ thông qua giao diện phần mềm như một người dùng thông thường (Hộp đen), nhưng sau đó sẽ truy cập trực tiếp vào cơ sở dữ liệu, tệp tin hệ thống hoặc theo dõi bộ nhớ để đối chiếu và xác minh xem dữ liệu thực tế (Actual) có được lưu trữ và cập nhật trạng thái chính xác với mong đợi (Expected) hay không (Hộp trắng).

2.3.4 Kiểm thử tự động (Automation Test)

Kiểm thử tự động là quá trình sử dụng các phần mềm, công cụ độc lập hoặc các đoạn mã kịch bản (scripts) để thực thi các bài kiểm thử một cách tự động. Phương pháp này có khả năng tự động nạp khối lượng lớn dữ liệu đầu vào, thực thi thao tác và so sánh kết quả tự động, giúp tối ưu hóa và rút ngắn đáng kể thời gian cho quy trình kiểm thử hồi quy (Regression Test) mỗi khi hệ thống có sự thay đổi hoặc cập nhật mã nguồn.

Chương 3

Phân tích và Thiết kế hệ thống

3.1 Kiến trúc tổng thể

Hệ thống được tổ chức theo mô hình kiến trúc bốn tầng (Four-Layer Architecture), với ranh giới trách nhiệm rõ ràng giữa các tầng và giao tiếp một chiều từ tầng trên xuống tầng dưới. Thiết kế này đảm bảo tính module hóa cao, giúp từng thành phần có thể được phát triển, kiểm thử và thay thế độc lập.

Tầng	Tên tầng	Thành phần và trách nhiệm
1	Presentation Layer (Giao diện)	Frontend ReactJS + Vite; gửi HTTP request đến API server qua localhost:8080; nhận JSON response và hiển thị cho người dùng.
2	API Gateway Layer (Cổng API)	main.cpp với Crow/ASIO; định tuyến các REST endpoint (/recommend, /products, /login, v.v.); xử lý CORS middleware; serialization/deserialization JSON.
3	Core Engine Layer (Thuật toán)	RatingMatrix, SimilarityMatrix, Recommender; thực thi Item-based CF + Cosine Similarity; UserManager, ItemManager, InteractionManager xử lý nghiệp vụ.
4	Data Layer (Dữ liệu)	CSVLoader đọc/ghi tệp CSV; DatabaseManager kết nối MySQL; các file .csv và schema MySQL là nguồn dữ liệu gốc.

Bảng 3.1: Kiến trúc bốn tầng của hệ thống

3.2 Phân tích yêu cầu và Use Case

3.2.1 Danh sách các tác nhân (Actors)

Hệ thống xác định hai tác nhân chính:

- Khách truy cập (Guest):** Người dùng chưa đăng nhập, chỉ có quyền xem danh sách sản phẩm và nhận gợi ý Cold Start (sản phẩm thịnh hành).
- Người dùng đã xác thực (Authenticated User):** Người dùng đã đăng nhập, được cá nhân hóa toàn bộ: nhận gợi ý theo lịch sử tương tác cá nhân, quản lý giỏ hàng và đặt hàng.

3.2.2 Bảng Use Case tổng hợp

UC ID	Tên Use Case	Actor	Mô tả ngắn
UC-01	Đăng ký tài khoản	Guest	Tạo tài khoản mới với họ tên, email, mật khẩu; hệ thống sinh <code>user_id</code> và lưu vào bộ nhớ + CSV.
UC-02	Đăng nhập	Guest	Xác thực bằng email + mật khẩu; hệ thống trả về token phiên làm việc.
UC-03	Đăng xuất	User	Hủy token phiên làm việc hiện tại.
UC-04	Xem trang chủ (sản phẩm thịnh hành)	Guest/User	Lấy Top 20 sản phẩm có điểm tổng hợp cao nhất; với User đã đăng nhập, hiển thị gợi ý cá nhân hóa.
UC-05	Tìm kiếm sản phẩm	Guest/User	Tìm kiếm theo từ khóa trong tên sản phẩm (case-insensitive substring).
UC-06	Lọc và sắp xếp sản phẩm	Guest/User	Lọc theo danh mục, nhãn (badge), điểm sao; sắp xếp theo giá hoặc mức độ bán chạy.
UC-07	Xem chi tiết sản phẩm	Guest/User	Xem thông tin đầy đủ của một sản phẩm; hệ thống ghi nhận hành vi click.
UC-08	Nhận gợi ý sản phẩm cá nhân	User	Hệ thống tính toán Item-based CF và trả về Top-N sản phẩm chưa tương tác.
UC-09	Nhận gợi ý Cold Start	Guest	Hệ thống trả về Top-N sản phẩm phổ biến nhất theo tổng điểm toàn hệ thống.
UC-10	Thêm vào giỏ hàng	User	Thêm sản phẩm với số lượng vào giỏ; nếu đã có thì cộng dồn.
UC-11	Cập nhật số lượng giỏ hàng	User	Thay đổi số lượng một sản phẩm trong giỏ; nếu số lượng = 0 thì xóa.
UC-12	Xóa sản phẩm khỏi giỏ	User	Loại bỏ hoàn toàn một mặt hàng khỏi giỏ hàng.
UC-13	Xem giỏ hàng	User	Lấy danh sách sản phẩm và tổng tiền trong giỏ hiện tại.
UC-14	Tạo đơn hàng	User	Chuyển toàn bộ giỏ hàng thành đơn, ghi nhận phương thức thanh toán; giỏ hàng được xóa sau khi tạo đơn thành công.
UC-15	Gợi ý chéo sau mua (Cross-sell)	User	Sau khi đặt hàng, hệ thống đề xuất tối đa 5 sản phẩm cùng danh mục nhưng chưa có trong đơn.
UC-16	Ghi nhận tương tác	User	Ghi log hành vi (click, add_cart, purchase) vào bộ nhớ RAM phục vụ cập nhật ma trận tương tác.

Bảng 3.2: Bảng Use Case tổng hợp của hệ thống

3.2.3 Luồng Use Case chính: Gợi ý sản phẩm cá nhân hóa (UC-08)

Điều kiện tiên quyết: Người dùng đã đăng nhập; `SimilarityMatrix` đã được xây dựng từ dữ liệu CSV khi khởi động server.

Luồng thực thi chính:

1. ReactJS gửi `GET /recommend/<user_id>` đến API server.
2. Crow route handler gọi `engine.forUser(user_id, 20)`.
3. `Recommender::forUser()` kiểm tra `rm_.isNewUser(uid)`.

4. Nếu user **đã tồn tại**: duyệt các item user đã tương tác, lấy danh sách hàng xóm từ `sm_.topK()`, tích lũy điểm dự đoán, Bubble Sort giảm dần, cắt Top-N.
5. Nếu user **chưa tồn tại**: gọi `coldStart(topN)` - trả về sản phẩm phổ biến nhất.
6. Kết quả (danh sách `item_id`) được serialize thành JSON và trả về ReactJS.
7. ReactJS gọi GET `/product/<item_id>` để lấy thông tin chi tiết và hiển thị.

Luồng ngoại lệ: Nếu `user_id` hợp lệ nhưng chưa có tương tác nào, luồng tự động fallback về Cold Start mà không báo lỗi cho người dùng.

3.3 Luồng hoạt động của hàm `main()`

Khi chương trình khởi động, hàm `main()` thực hiện tuần tự các bước sau:

1. **Nạp dữ liệu (Load CSV):** `CSVLoader` đọc ba file `users.csv`, `items.csv`, `interactions.csv` từ thư mục `4_dataset/`, chuyển đổi thành các đối tượng `MyVector<User>`, `MyVector<Item>`, `MyVector<Interaction>`.
2. **Đồng bộ MySQL (DB Sync):** `DatabaseManager` kết nối đến MySQL Server, tự động tạo bảng nếu chưa tồn tại (`createTables()`), sau đó import toàn bộ sản phẩm qua `insertItem()`.
3. **Xây dựng Rating Matrix:** `RatingMatrix::buildMatrix()` duyệt qua `MyVector<Interaction>` và tính điểm tổng hợp cho mỗi cặp (user, item) theo công thức:

$$\text{Score} = \text{click} \times 1,0 + \text{add_cart} \times 2,0 + \text{purchase} \times 5,0 + \text{rating} \times 3,0$$
4. **Tính Similarity Matrix:** `SimilarityMatrix::build()` đảo chuyển (transpose) ma trận sang Item-User rồi tính Cosine Similarity cho mọi cặp sản phẩm, lưu vào `MyMap<string, MyMap<string, double>> simMatrix`.
5. **Khởi tạo các Manager:** Các đối tượng `UserManager`, `ItemManager`, `InteractionManager` và `Recommender` được khởi tạo với dữ liệu đã nạp.
6. **Khởi động API Server:** `crow::App` đăng ký các route, sau đó server được chạy trên luồng riêng (`std::thread`) để không chặn luồng demo console.
7. **Chạy demo console:** Thực thi tuần tự các demo: Auth (đăng ký/đăng nhập/đăng xuất), Item (tìm kiếm/lọc/chi tiết), Cart & Order (giỏ hàng/đặt hàng/cross-sell).

3.4 Thiết kế lớp (Class Design)

3.4.1 Tầng Thực thể (1_models)

Ba lớp thực thể đóng vai trò là nền tảng dữ liệu của toàn hệ thống. Tất cả các thành phần tầng trên đều phụ thuộc vào ba lớp này, trong khi chúng không phụ thuộc vào bất kỳ module nào khác - đây là nguyên lý phụ thuộc một chiều quan trọng trong kiến trúc phân tầng.

Lớp	Thuộc tính chính	Phương thức quan trọng
User	<code>user_id</code> , <code>name</code> , <code>email</code> , <code>password</code> , <code>created_at</code>	<code>checkPassword(pwd)</code> xác thực mật khẩu; các getter/setter theo chuẩn encapsulation.
Item	<code>item_id</code> , <code>name</code> , <code>category</code> , <code>badge</code> , <code>price</code> , <code>star</code> , <code>sold</code> , <code>image_url</code>	<code>displayInfo()</code> in thông tin; getter/setter đầy đủ cho tất cả thuộc tính.
Interaction	<code>user_id</code> , <code>item_id</code> , <code>click_count</code> , <code>add_cart_count</code> , <code>purchase_count</code> , <code>rating</code>	<code>computeScore()</code> tính điểm tổng hợp có trọng số - đây là hàm cốt lõi của tầng thực thể.

Bảng 3.3: Thiết kế các lớp thực thể (1_models)

Thiết kế hàm `computeScore()`: Đây là điểm phân biệt then chốt so với các hệ thống chỉ dùng rating số sao đơn thuần. Hàm kết hợp bốn tín hiệu hành vi với trọng số phản ánh mức độ cam kết thực sự:

Hành vi	Trọng số	Hằng số	Lý do thiết kế
Click (xem sản phẩm)	1.0	WEIGHT_CLICK	Tín hiệu nhẹ nhất: người dùng chỉ tò mò
Thêm vào giỏ hàng	2.0	WEIGHT_ADD_CART	Có ý định mua nhưng chưa quyết định
Mua hàng	5.0	WEIGHT_PURCHASE	Cam kết cao nhất: đã chi tiền thực sự
Đánh giá (rating)	3.0	WEIGHT_RATING	Phản hồi có chủ đích, nhân với giá trị sao

Bảng 3.4: Bảng trọng số hành vi trong hàm `computeScore()`

Công thức tổng hợp:

$$\text{Score} = (\text{click} \times 1.0) + (\text{add_cart} \times 2.0) + (\text{purchase} \times 5.0) + (\text{rating} \times 3.0)$$

3.4.2 Tầng Hạ tầng (2_data)

- **CSVLoader:** Đọc/ghi ba file CSV với xử lý chuỗi an toàn qua `trim()` (loại bỏ khoảng trắng/CRLF) và `escapeField()` (escape dấu phẩy và ngoặc kép theo chuẩn RFC 4180). Mỗi phương thức `loadXxx()` bỏ qua dòng header, parse từng trường bằng `std::getline(ss, field, ',')`, và push đối tượng vào `MyVector` nội bộ.
- **DatabaseManager:** Bọc MySQL Connector, cung cấp phương thức `connect()`, `createTables()`, `insertUser/Item/Interaction()` - thêm dữ liệu, `loadUsers/Items/Interactions()` - nạp dữ liệu, `deleteUser/Item/Interaction()` - xóa dữ liệu. Sử dụng `mysql_real_escape_string()` để chống SQL Injection. Chiến lược `ON DUPLICATE KEY UPDATE` cho phép import lại CSV nhiều lần mà không gây lỗi trùng khóa.
- **MyDataStructures.h:** Định nghĩa `MyVector<T>`, `MyMap<K,V>`, `MyPair<K,V>` thay thế hoàn toàn STL. File header duy nhất này là nền tảng cho toàn bộ hệ thống.

3.4.3 Tầng Thuật toán (3_core)

- **RatingMatrix:** Lưu trữ ma trận thưa dưới dạng `MyMap<string, MyMap<string,double>>` (map lồng nhau: `user_id` $\rightarrow$ `item_id` $\rightarrow$ `score`). Phương thức của `buildMatrix()` duyệt qua `MyVector<Interaction>`, gọi `computeScore()` cho từng bản ghi. Phương thức `normalizeMatrix()` thực hiện chuẩn hóa Mean-Centering, giúp giảm ảnh hưởng của xu hướng đánh giá cao/thấp cá nhân. Phương thức `itemTotalScore()` tổng hợp điểm tất cả người dùng cho một sản phẩm, phục vụ Cold Start.
- **SimilarityMatrix:** Lưu ma trận tương đồng item-item dưới dạng `MyMap<string, MyMap<string,double>> simMatrix`. Phương thức `build()` thực hiện hai bước: (1) đảo chuyển ma trận từ User-Item sang Item-User, (2) duyệt mọi cặp item (i, j) với $j > i$ để tránh tính trùng, gọi `calculateCosine()` và lưu kết quả đối xứng. Kỹ thuật `smallerMap` trong `calculateCosine()` tối ưu hóa hiệu năng trên ma trận thưa.
- **Recommender:** Điều phối `forUser(user_id, topN)` (gợi ý cá nhân hóa) và `coldStart(topN)` (gợi ý phổ thông cho người dùng mới). Hàm `printResult()` hiển thị kết quả ra console dạng bảng.

3.4.4 Tầng Nghiệp vụ (6_services)

- **UserManager:** Quản lý `MyVector<User>` nội bộ và `MyMap<string,string> sessions` (`user_id` $\rightarrow$ `token`). Phương thức `Register()` kiểm tra email trùng lặp trước khi tạo tài khoản mới với `user_id` sinh tự động (tìm max số hiện tại và tăng lên 1). Phương thức `Login()` tìm kiếm email, xác thực mật khẩu, tạo token ngẫu nhiên dựa trên `time()` và `rand()`.
- **ItemManager:** `SearchProducts(keyword)` thực hiện tìm kiếm substring không phân biệt hoa thường qua `toLowerCaseStr()`. `FilterAndSortProducts()` áp dụng bộ lọc đa tiêu chí (category, badge, minStar) rồi sắp xếp bằng Bubble Sort theo một trong bốn tiêu chí: `price_asc`, `price_desc`, `star`, `sold`.
- **InteractionManager:** Quản lý hai `MyMap` nội bộ: `carts[userId] = MyVector<CartItem>` và `orders[orderId] = Order`. Phương thức `AddToCart()` kiểm tra xem sản phẩm đã có trong giỏ chưa trước khi thêm/cộng dồn. `CreateOrder()` tổng hợp giỏ hàng thành đơn và xóa giỏ.

`GetSuggestedProducts()` thực hiện cross-sell bằng cách tìm sản phẩm cùng danh mục với đơn hàng vừa tạo.

3.5 Thiết kế quan hệ dữ liệu

3.5.1 Quan hệ giữa các thực thể

Thực thể nguồn	Kiểu quan hệ	Thực thể đích	Mô tả
User	1 - N	Interaction	Một người dùng có thể có nhiều bản ghi tương tác với các sản phẩm khác nhau.
Item	1 - N	Interaction	Một sản phẩm có thể xuất hiện trong nhiều bản ghi tương tác của nhiều người dùng khác nhau.
User	N - M	Item	Quan hệ nhiều-nhiều được thể hiện thông qua bảng Interaction trung gian.
CartItem	N - 1	Order	Nhiều CartItem thuộc về một Order; sau khi CreateOrder, giỏ hàng bị xóa.
Item	1 - N	CartItem	Một Item có thể xuất hiện trong CartItem của nhiều người dùng đồng thời.

Bảng 3.5: Quan hệ giữa các thực thể trong hệ thống

3.5.2 Thiết kế cơ sở dữ liệu CSV

Dữ liệu được lưu trong ba file CSV phẳng tại thư mục `4_dataset/`:

File	Các trường	Kích thước mẫu
<code>users.csv</code>	<code>user_id</code> , <code>name</code> , <code>created_at</code> , <code>email</code> , <code>password</code>	10.000 bản ghi
<code>items.csv</code>	<code>item_id</code> , <code>name</code> , <code>category</code> , <code>price</code>	500 bản ghi
<code>interactions.csv</code>	<code>user_id</code> , <code>item_id</code> , <code>click</code> , <code>add_cart</code> , <code>purchase</code> , <code>rating</code>	100.000 bản ghi

Bảng 3.6: Cấu trúc các file CSV dữ liệu

Quy tắc xử lý chuỗi trong CSV: trường chứa dấu phẩy hoặc dấu nhảy kép sẽ được bao bởi dấu nhảy kép (`escapeField()`); dấu nhảy kép bên trong được nhân đôi. Dòng header được bỏ qua khi đọc (`getline` đầu tiên). Khoảng trắng và ký tự `CR/LF` ở đầu/cuối trường được loại bỏ bằng `trim()`.

3.5.3 Lược đồ MySQL

Bảng	Cột (Kiểu)	Ràng buộc	Ghi chú
users	user_id VARCHAR(20)	PRIMARY KEY	Khóa chính
	name VARCHAR(100)	NOT NULL	
	created_at DATE		
items	item_id VARCHAR(20)	PRIMARY KEY	Khóa chính
	name VARCHAR(150)	NOT NULL	
	category VARCHAR(50)		
	price DOUBLE	NOT NULL DEFAULT 0	
interactions	id INT	PRIMARY KEY, AUTO_INCREMENT	
	user_id VARCHAR(20)	FOREIGN KEY → users	ON DELETE CASCADE
	item_id VARCHAR(20)	FOREIGN KEY → items	ON DELETE CASCADE
	click, add_cart, purchase INT	NOT NULL DEFAULT 0	
	rating DOUBLE	NOT NULL DEFAULT 0	

Bảng 3.7: Lược đồ MySQL của hệ thống

3.6 Thiết kế API Endpoints

Method	Endpoint	Luồng xử lý	Response JSON
GET	/products	DatabaseManager.loadItems()	{products: [...]}
GET	/trending	Tính tổng điểm, Bubble Sort	{trending_items: [...]}
GET	/recommend/<uid>	Recommender.forUser()	{recommendations: [...]}
GET	/product/<id>	ItemManager.GetProductDetail()	{product: {...}}
POST	/register	UserManager.Register()	{status, user_id}
POST	/login	UserManager.Login()	{status, user_id, token}
POST	/logout	UserManager.Logout()	{status, message}
POST	/cart/add	InteractionManager.AddToCart()	{status, message}
POST	/cart/remove	InteractionManager.RemoveFromCart()	{status, message}
POST	/cart/get	InteractionManager.GetCartDetails()	{items: [...], total}
POST	/order/create	InteractionManager.CreateOrder()	{status, order_id}
POST	/track	InteractionManager.AddInteraction()	{status, message}

Bảng 3.8: Bảng thiết kế REST API Endpoints

Tất cả các endpoint đều được xử lý CORS đầy đủ thông qua cors_middleware: mỗi response tự động thêm header Access-Control-Allow-Origin: * và hỗ trợ OPTIONS preflight request (trả về 204 No Content).

Chương 4

Cài đặt hệ thống

4.1 Cấu trúc thư mục chương trình thực tế

Cây thư mục của hệ thống được tổ chức theo nguyên tắc phân chia module theo chức năng (modularization by feature), một cách tiếp cận phổ biến trong các dự án phần mềm quy mô vừa và lớn. Mỗi thư mục cấp một (từ `1_models` đến `6_services`) tương ứng với một tầng kiến trúc hoặc một nhóm chức năng cụ thể, giúp lập trình viên dễ dàng định vị mã nguồn cần sửa đổi mà không phải tìm kiếm qua hàng trăm tệp rải rác. Các thư mục được đánh số thứ tự (`1_`, `2_`, `3_`,..., `6_`) thể hiện thứ tự phụ thuộc tăng dần - module `1_models` không phụ thuộc vào module nào, trong khi module `6_services` và `3_core` phụ thuộc vào `1_models` và `2_data`.

```

Recommendation-System/
├── main.cpp ..... Điểm vào chương trình (API Controller)
├── README.md ..... Tài liệu hướng dẫn
├── SETUP_MYSQL_CONNECTOR.md ..... Hướng dẫn cấu hình MySQL
├── 1_models/ ..... [Tầng Thực thể] Định nghĩa cấu trúc dữ liệu
│   ├── User.h / User.cpp ..... Lớp Người dùng
│   ├── Item.h / Item.cpp ..... Lớp Sản phẩm
│   └── Interaction.h / Interaction.cpp ..... Lớp Tương tác
├── 2_data/ ..... [Tầng Hạ tầng] Quản lý dữ liệu
│   ├── MyDataStructures.h ..... Cấu trúc dữ liệu tự định nghĩa (MyVector, MyMap, MyPair)
│   ├── CSVLoader.h / CSVLoader.cpp ..... Bộ nạp dữ liệu từ tệp CSV
│   ├── DatabaseManager.h / .cpp ..... Quản lý kết nối MySQL
│   ├── DataCreator.py ..... Script Python sinh dữ liệu mẫu
│   └── db_config.example.h ..... Mẫu cấu hình kết nối cơ sở dữ liệu
├── 3_core/ ..... [Tầng Thuật toán] Engine gợi ý
│   ├── RatingMatrix.h / .cpp ..... Ma trận tương tác User-Item
│   ├── SimilarityMatrix.h / .cpp ..... Ma trận độ tương đồng Cosine Item-Item
│   └── Recommender.h / Recommender.cpp ..... Bộ tổng hợp gợi ý (Item-based CF + Cold Start)
├── 4_dataset/ ..... Dữ liệu thô (CSV & SQL)
│   ├── users.csv ..... Dữ liệu người dùng
│   ├── items.csv ..... Dữ liệu sản phẩm
│   ├── interactions.csv ..... Dữ liệu tương tác
│   ├── recommendationsystem_users.sql
│   ├── recommendationsystem_items.sql
│   └── recommendationsystem_interactions.sql
├── 6_services/ ..... [Tầng Nghiệp vụ] Logic xử lý
│   ├── UserManager.h / .cpp ..... Quản lý người dùng (Auth)
│   ├── ItemManager.h / .cpp ..... Quản lý sản phẩm (Search, Filter)
│   └── InteractionManager.h / .cpp ..... Quản lý tương tác (Cart, Order)

```

4.2 Triển khai xây dựng các hàm chính

Trong khuôn khổ đề án này, nhóm phát triển lựa chọn không sử dụng các thư viện chuẩn STL (Standard Template Library) của C++ nhằm mục đích học thuật, giúp người học hiểu sâu về cơ chế hoạt động bên trong của các cấu trúc dữ liệu phổ biến. Thay vào đó, hệ thống triển khai hai cấu trúc dữ liệu nền tảng: `MyVector` (mảng động) và `MyMap` (bảng băm), được định nghĩa trong tệp `2_data/MyDataStructures.h`.

4.2.1 Cấu trúc cặp giá trị `MyPair` (Cài đặt thay thế `std::pair`)

`MyPair` là một struct template (`template <typename K, typename V>`) đóng vai trò là kiểu dữ liệu trung gian dùng để biểu diễn một cặp khóa-giá trị (key-value pair). Struct này được sử dụng xuyên suốt hệ thống, đặc biệt là trong kiểu `RecommendResult` - định nghĩa là `MyVector<MyPair<std::string, double>` dùng để lưu trữ danh sách các item cùng với điểm dự đoán tương ứng.

`MyPair` đóng vai trò then chốt trong việc kết nối các cấu trúc dữ liệu khác:

- Là kiểu phần tử của `MyVector` trong `RecommendResult` (kết quả gợi ý).
- Là kiểu trả về của iterator trong `MyMap` thông qua phương thức `operator*()`.
- Là kiểu dữ liệu trong `SimilarityMatrix` khi biểu diễn danh sách hàng xóm gần nhất dưới dạng `MyVector<MyPair<std::string, double>`.

Cấu trúc bộ nhớ: `MyPair` sở hữu hai trường dữ liệu public:

- `K first` - Khóa (key), trong ngữ cảnh hệ thống thường là `std::string` (mã định danh của item hoặc người dùng).
- `V second` - Giá trị (value), trong ngữ cảnh hệ thống thường là `double` (điểm dự đoán hoặc độ tương đồng).

Đoạn mã dưới đây minh họa cách khai báo và các constructor của `MyPair` trong file `MyDataStructures.h`:

```

1  template <typename K, typename V>
2  struct MyPair {
3      K first;
4      V second;
5
6      MyPair() : first(), second() {}
7      MyPair(const K& f, const V& s) : first(f), second(s) {}
8      MyPair(const MyPair& other) : first(other.first), second(other.
second) {}
9
10     MyPair& operator=(const MyPair& other) {
11         if (this != &other) {
12             first = other.first;
13             second = other.second;
14         }
15         return *this;
16     }
17
18     bool operator==(const MyPair& other) const {
19         return first == other.first && second == other.second;
20     }
21 };

```

Listing 4.1: Cấu trúc `MyPair` (File `MyDataStructures.h`)

Toán tử gán và so sánh:

- **Copy Assignment Operator** (`operator=`): Gán lần lượt `first` và `second` từ đối tượng nguồn. Có kiểm tra tự gán (`this != &other`) để đảm bảo an toàn.
- **Equality Operator** (`operator==`): So sánh hai đối tượng `MyPair` bằng nhau nếu cả `first` và `second` đều bằng nhau tương ứng. Toán tử này được sử dụng trong các thao tác tìm kiếm và kiểm tra phần tử.

4.2.2 Cấu trúc MyVector (Cài đặt thay thế std::vector)

MyVector là một lớp template (`template <typename T>`) quản lý một mảng cấp phát động trên heap, cho phép lưu trữ một dãy các phần tử kiểu T và tự động mở rộng sức chứa khi cần thiết.

Cấu trúc bộ nhớ:

- **T* data** - Con trỏ trỏ tới vùng nhớ trên heap chứa các phần tử.
- **int sz** - Số lượng phần tử hiện tại (**size**).
- **int cap** - Sức chứa tối đa hiện tại (**capacity**), tức số phần tử tối đa có thể chứa trước khi cần cấp phát lại.

Cơ chế cấp phát động và mở rộng: Khi một phần tử mới được thêm vào thông qua phương thức `push_back(const T& value)`, hệ thống kiểm tra điều kiện `sz >= cap`. Nếu vùng đệm hiện tại đã đầy, một vùng nhớ mới với sức chứa gấp đôi được cấp phát thông qua hàm `reserveMore()`:

```

1 void push_back(const T& value) {
2     if (sz >= cap) {
3         int new_cap = (cap == 0) ? 4 : cap * 2;
4         reserveMore(new_cap);
5     }
6     data[sz] = value;
7     ++sz;
8 }
9
10 void reserveMore(int new_cap) {
11     T* new_data = new T[new_cap];
12     for (int i = 0; i < sz; ++i) {
13         new_data[i] = data[i];
14     }
15     delete[] data;
16     data = new_data;
17     cap = new_cap;
18 }

```

Listing 4.2: Hàm `push_back` và `reserveMore` (File `MyDataStructures.h`)

Ba quy tắc lớn - Phòng chống rò rỉ bộ nhớ:

1. **Destructor** (`~MyVector`): Giải phóng toàn bộ vùng nhớ do `data` quản lý bằng `delete[] data`. Gán `data = nullptr`, `sz = 0`, `cap = 0` để tránh dangling pointer.
2. **Copy Constructor** (`MyVector(const MyVector& other)`): Tạo một bản sao độc lập (deep copy) bằng cách cấp phát một mảng mới có cùng sức chứa `cap = other.cap`, sau đó sao chép từng phần tử từ `other.data` sang `data`.
3. **Copy Assignment Operator** (`operator=`): Giải phóng vùng nhớ hiện tại, sau đó thực hiện cấp phát mới và sao chép toàn bộ dữ liệu từ đối tượng nguồn. Kiểm tra tự gán (`this != &other`) để tránh lỗi hủy dữ liệu trước khi sao chép.

Một số thao tác cơ bản:

Thao tác	Độ phức tạp	Giải thích
<code>operator[] / at(index)</code>	$O(1)$	Truy cập trực tiếp qua địa chỉ bộ nhớ <code>data + index</code> , không phụ thuộc vào kích thước mảng.
<code>push_back</code> (thường)	$O(1)$	Thao tác thêm vào vị trí cuối cùng được thực hiện trong thời gian hằng số.
<code>push_back</code> (khi đầy)	$O(n)$	Hệ thống cấp phát vùng nhớ mới gấp đôi, sao chép toàn bộ n phần tử cũ.
<code>push_back</code> (khấu hao)	$O(1)^*$	Độ phức tạp trung bình là hằng số nhờ chiến lược nhân đôi sức chứa.
<code>pop_back</code>	$O(1)$	Chỉ giảm biến đếm <code>size</code> , không cần giải phóng bộ nhớ.
<code>clear</code>	$O(n)$	Thu hồi toàn bộ vùng nhớ đang sử dụng.

Bảng 4.1: Các thao tác của MyVector và độ phức tạp

Ghi chú: Ký hiệu $O(1)^*$ biểu thị chi phí khấu hao. Mặc dù một lần `push_back` riêng lẻ có thể tốn $O(n)$ khi cần mở rộng, tổng chi phí của N lần `push_back` liên tiếp là $O(N)$, dẫn đến chi phí trung bình mỗi lần là $O(1)$.

4.2.3 Bảng băm MyMap (Cài đặt thay thế `std::unordered_map`)

MyMap là một lớp template (`template <typename K, typename V>`) hiện thực cấu trúc dữ liệu từ điển (dictionary) lưu trữ các cặp khóa-giá trị (key-value pairs). Cấu trúc này sử dụng bảng băm (hash table) với phương pháp Separate Chaining để xử lý va chạm.

MyMap sở hữu ba thành viên dữ liệu chính:

- `Node** buckets` - Mảng các con trỏ, mỗi phần tử trỏ tới đầu một danh sách liên kết đơn (singly linked list) chứa các nút có cùng chỉ số băm.
- `int bucket_count` - Số lượng bucket (mặc định khởi tạo là 16).
- `int sz` - Tổng số cặp key-value đang được lưu trữ.

Mỗi nút trong danh sách liên kết được định nghĩa là một struct `Node`:

```

1 struct Node {
2     K key;
3     V value;
4     Node* next;
5     Node(const K& k, const V& v) : key(k), value(v), next(nullptr) {}
6 };

```

Listing 4.3: Cấu trúc Node bên trong MyMap (File MyDataStructures.h)

Cơ chế băm: Sử dụng lớp `MyHashHelper` với các template specialization cho từng kiểu dữ liệu:

- Với `std::string`: Duyệt từng ký tự, nhân h với 31 và cộng giá trị ASCII ($h = h * 31 + (\text{unsigned char})\text{key}[i]$).
- Với `int`: Lấy giá trị tuyệt đối và ép kiểu thành `unsigned long`.
- Với `double`: Lấy giá trị tuyệt đối, nhân với 1000 và ép kiểu thành `unsigned long`.
- Với kiểu tổng quát `T`: Đọc từng byte của đối tượng và áp dụng cùng công thức băm.

Nén (compression): Chỉ số bucket được tính bằng phép toán `hashValue(key) % bucket_count`.

Xử lý va chạm bằng Separate Chaining: Khi xảy ra va chạm băm, hệ thống tạo một nút mới và nối nó vào ngay trước (đầu) các nút đã có sẵn trong danh sách liên kết của ô đó:

```

1 void insert(const K& key, const V& value) {
2     unsigned long idx = hash(key);
3     Node* cur = buckets[idx];
4

```

```

5      // Kiểm tra key đã tồn tại chưa
6      while (cur != nullptr) {
7          if (cur->key == key) {
8              cur->value = value; // Cập nhật value
9              return;
10         }
11         cur = cur->next;
12     }
13
14     // Thêm nút mới vào đầu danh sách
15     Node* new_node = new Node(key, value);
16     new_node->next = buckets[idx];
17     buckets[idx] = new_node;
18     ++sz;
19 }

```

Listing 4.4: Thuật toán insert của MyMap (File MyDataStructures.h)

Cơ chế quản lý bộ nhớ: Tương tự MyVector, MyMap tuân thủ Ba quy tắc lớn:

- **Destructor:** Duyệt toàn bộ các bucket, xóa lần lượt từng nút trong danh sách liên kết (hàm `clearBuckets()`), sau đó giải phóng mảng `buckets`.
- **Copy Constructor:** Tạo bảng băm mới với cùng số bucket, sau đó duyệt toàn bộ các nút của bảng gốc và gọi `insert()` cho từng cặp key-value.
- **Copy Assignment Operator:** Giải phóng bảng hiện tại, tạo bảng mới và sao chép toàn bộ dữ liệu.

Một số thao tác cơ bản:

Thao tác	Trường hợp trung bình	Trường hợp xấu nhất	Giải thích
<code>insert</code>	$O(1)$	$O(n)$	Trung bình: Hàm băm phân phối đều. Xấu nhất: Tất cả khóa rơi vào cùng một bucket.
<code>find</code>	$O(1)$	$O(n)$	Tương tự thao tác chèn, cần duyệt danh sách tại bucket đã được băm tới.
<code>contains</code>	$O(1)$	$O(n)$	Kiểm tra sự tồn tại của khóa thông qua việc tìm kiếm danh sách liên kết.
<code>operator[]</code>	$O(1)$	$O(n)$	Có thể kích hoạt tạo mới nút nếu khóa chưa tồn tại.
<code>remove</code>	$O(1)$	$O(n)$	Cần thực hiện tìm kiếm phần tử trước khi tách nút ra khỏi danh sách.

Bảng 4.2: Phân tích độ phức tạp thời gian của các thao tác trên Hash Map

Giải thích: Trong trường hợp xấu nhất, hàm băm có thể bị suy thoái - tất cả các khóa đều cho cùng một chỉ số băm. Khi đó, toàn bộ dữ liệu tập trung vào một danh sách liên kết duy nhất, biến bảng băm thành danh sách liên kết với độ phức tạp $O(n)$ cho mọi thao tác. Trong thực tế, với hàm băm phân phối đều và hệ số tải (load factor) hợp lý, hầu hết các thao tác đạt $O(1)$.

4.3 Các thuật toán nền tảng

4.3.1 Thuật toán Sắp xếp nổi bọt (Bubble Sort)

Sau khi hệ thống tính toán điểm dự đoán (prediction score) cho từng item, cần sắp xếp danh sách kết quả theo thứ tự giảm dần để lấy topN item có điểm cao nhất gợi ý cho người dùng. Thuật toán được lựa chọn cho tác vụ này là Bubble Sort (Sắp xếp nổi bọt).

```

1 // Sắp xếp giảm dần theo score (Bubble Sort)
2 for (int i = 0; i < result.size() - 1; ++i) {
3     for (int j = 0; j < result.size() - i - 1; ++j) {
4         if (result[j].second < result[j + 1].second) {
5             MyPair<std::string, double> temp = result[j];
6             result[j] = result[j + 1];
7             result[j + 1] = temp;
8         }
9     }
10 }
```

Listing 4.5: Thuật toán Bubble Sort trong Recommender::forUser() (File Recommender.cpp)

Bubble Sort được triển khai tại hai vị trí trong tệp `3_core/Recommender.cpp`:

1. Hàm `forUser()` (dòng 62–71): Sắp xếp danh sách `RecommendResult` - tức `MyVector<MyPair<std::string, double>` - chứa các item được gợi ý cho người dùng đã có lịch sử tương tác.
2. Hàm `coldStart()` (dòng 91–99): Sắp xếp danh sách các item cho người dùng mới (chưa có tương tác), dựa trên tổng điểm toàn cục của item.

Cả hai đoạn mã đều có cùng cấu trúc thuật toán giống nhau.

Độ phức tạp thời gian:

Trường hợp	Số lần so sánh	Số lần hoán đổi	Big O	Giải thích
Tốt nhất (Best Case)	$n(n-1)/2$	0	$O(n^2)$	Dù mảng đã sắp xếp, thuật toán vẫn thực hiện toàn bộ vòng lặp vì không có cơ chế dừng sớm (flag).
Trung bình (Average Case)	$n(n-1)/2$	$\approx n(n-1)/4$	$O(n^2)$	Với dữ liệu ngẫu nhiên, trung bình khoảng một nửa số cặp phần tử cần thực hiện hoán đổi.
Xấu nhất (Worst Case)	$n(n-1)/2$	$n(n-1)/2$	$O(n^2)$	Xảy ra khi mảng sắp xếp ngược chiều mong muốn, mỗi lần so sánh đều dẫn đến hoán đổi.

Bảng 4.3: Phân tích độ phức tạp thời gian của Bubble Sort

Ghi chú: Công thức tính tổng số lần so sánh:

$$\text{Tổng số lần so sánh} = (n-1) + (n-2) + \cdots + 2 + 1 = \frac{n(n-1)}{2}$$

Do đó $T(n) \approx n^2/2$, suy ra độ phức tạp là $O(n^2)$.

Độ phức tạp không gian:

Loại	Big O	Giải thích
Bộ nhớ phụ (Auxiliary Space)	$O(1)$	Thuật toán chỉ sử dụng duy nhất một biến tạm temp để hoán đổi giá trị. Chi phí bộ nhớ bổ sung là hằng số, không phụ thuộc vào n .
Tổng bộ nhớ (Total Space)	$O(n)$	Mảng đầu vào result chiếm $O(n)$ bộ nhớ để lưu trữ n phần tử kiểu MyPair , đây là không gian lưu trữ dữ liệu gốc.

Bảng 4.4: Phân tích độ phức tạp không gian của Bubble Sort

4.3.2 Triển khai hàm tính toán Cosine Similarity tối ưu bằng kỹ thuật `smallerMap`

Trong một hệ thống gợi ý thực tế, ma trận tương tác User-Item hầu như luôn là một **ma trận thưa (Sparse Matrix)** - nghĩa là phần lớn các ô trong ma trận đều bằng 0, bởi một người dùng chỉ tương tác với một số rất ít sản phẩm trong tổng số hàng nghìn hoặc hàng triệu sản phẩm có sẵn. Ví dụ, với 10.000 người dùng và 5.000 sản phẩm, một người dùng trung bình chỉ tương tác với khoảng 20-50 sản phẩm, dẫn đến mật độ ma trận chỉ khoảng 0,1%-0,5%.

Nếu tính toán Cosine Similarity theo cách tuần tự - duyệt qua toàn bộ các `user_id` của cả hai vector mỗi lần tính - thì có tới hơn 99% phép nhân là thừa thãi (nhân với 0). Kỹ thuật `smallerMap` được triển khai trong hàm `SimilarityMatrix::calculateCosine()` là một bước tối ưu hóa then chốt giải quyết triệt để vấn đề này:

```

1 // Meo toi uu: Chi duyet qua map co kich thuoc nho hon
2 const MyMap<std::string, double>* smallerMap = &itemA_users;
3 const MyMap<std::string, double>* largerMap = &itemB_users;
4
5 if (itemA_users.size() > itemB_users.size()) {
6     smallerMap = &itemB_users;
7     largerMap = &itemA_users;
8 }
9
10 // Chi nhan diem cua nhung User da tuong tac voi CA HAI item
11 MyVector<std::string> smallerKeys = smallerMap->keys();
12 for (int i = 0; i < smallerKeys.size(); ++i) {
13     std::string user_id = smallerKeys[i];
14     double scoreA;
15     smallerMap->find(user_id, scoreA);
16
17     // Tim xem User nay co tuong tac voi item ben kia khong?
18     double scoreB;
19     if (largerMap->find(user_id, scoreB)) {
20         dotProduct += (scoreA * scoreB); // R_{u,i} * R_{u,j}
21     }
22 }

```

Listing 4.6: Kỹ thuật `smallerMap` trong `SimilarityMatrix::calculateCosine()` (File `SimilarityMatrix.cpp`)

Phân tích kỹ thuật cốt lõi:

- Phép tính tích vô hướng (tử số của Cosine) chỉ có ý nghĩa trên tập giao của hai vector (người dùng đã tương tác với cả hai sản phẩm). Phép so sánh kích thước giúp hệ thống luôn chọn ra `smallerMap` (vector chứa ít phần tử hơn) làm gốc để duyệt.
- Thay vì duyệt qua toàn bộ không gian người dùng (N_u), vòng lặp `for` chỉ duyệt qua các `user_id` thực sự có dữ liệu trong `smallerMap`. Với mỗi `user_id` này, hệ thống tận dụng tốc độ $O(1)$ của Bảng băm để tra cứu xem nó có tồn tại trong `largerMap` hay không. Việc này mang lại lợi ích kép: loại bỏ 100% các phép nhân vô ích với 0 và tránh việc phải duyệt mảng lớn.
- Tác động hiệu năng (Big O):** Gọi N_u là tổng số người dùng, d là mật độ tương tác trung bình (số user tương tác với 1 item).

Cách tiếp cận	Độ phức tạp 1 cặp item
Ngây thơ (duyệt toàn bộ user)	$O(N_u)$
Tối ưu <code>smallerMap</code>	$O(d)$

Bảng 4.5: So sánh độ phức tạp tính toán Cosine Similarity

Bằng kỹ thuật này, độ phức tạp thuật toán được giảm từ phụ thuộc vào tổng quy mô hệ thống $O(N_u)$ xuống chỉ còn phụ thuộc vào mật độ tương tác $O(d)$ (vốn rất nhỏ và ổn định). Nhờ đó, thuật toán cắt giảm được hàng tỷ phép tính thừa, giúp hệ thống tính toán ma trận mượt mà trên bộ nhớ RAM mà không bị tràn tải CPU.

Chương 5

Thực nghiệm và đánh giá kết quả

5.1 Môi trường thực nghiệm

Thành phần	Cấu hình
Hệ điều hành	Windows 11 x64
Trình biên dịch	MinGW-w64 g++ 13.1.0, C++17
Kích thước dữ liệu	10.000 users, 500 items, 100.000 interactions
Cơ sở dữ liệu	MySQL Server 8.0.46
Frontend	ReactJS + Vite (Node.js 20)

Bảng 5.1: Môi trường thực nghiệm

5.2 Kết quả chạy demo console

Sau khi build thành công và khởi chạy chương trình, hệ thống thực hiện tuần tự ba luồng demo trên console: xác thực người dùng (Auth), quản lý sản phẩm (Item), và quản lý giỏ hàng/đơn hàng (Cart & Order). Đồng thời, API server khởi động tại `localhost:8080` chạy song song trên luồng riêng.

[Chèn ảnh chụp màn hình kết quả console sau khi build và chạy chương trình.]

```

PS C:\Users\84911\Recommendation-System-1> .\recommendation.exe
>>
=====

[1] Đang đọc dữ liệu từ thư mục 4_dataset...
    Users: 9987 | Items: 500

[1.5] Đang đóng gói dữ liệu CSV vào MySQL...
[DatabaseManager] Đã kiểm tra/tạo xong các bảng users, items, interactions.
    => Nạp thành công 500 sản phẩm vào Database!

>>> Server API đã khởi động tại http://localhost:8080/recommend/<user_id>

=====
    AUTH: Đăng ký & Đăng nhập & Đăng xuất
=====

--- DANG KY ---
[+] Đăng ký thành công! User ID: U9991
    => Thành công! User ID: U9991

--- DANG NHAP ---
[+] Đăng nhập thành công! Xin chào Nguyen Van Teo
    => Token: TOKEN_1782234949_41

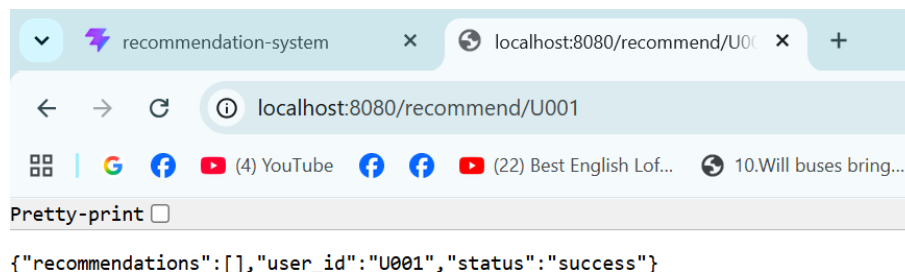
```

Hình 5.1: Kết quả chạy demo console - luồng Auth, Item và Cart/Order

5.3 Kết quả gợi ý sản phẩm

Giao diện ReactJS gọi API GET `/recommend/<user_id>` và hiển thị danh sách Top-20 sản phẩm được gợi ý. Với người dùng đã có lịch sử tương tác, kết quả phản ánh đúng các sản phẩm tương đồng với những gì họ đã tương tác. Với người dùng mới, hệ thống hiển thị danh sách sản phẩm thịnh hành (Cold Start).

[Chèn ảnh giao diện ReactJS hiển thị kết quả gợi ý từ API `/recommend/<user_id>`.]



Hình 5.2: Giao diện ReactJS hiển thị danh sách gợi ý sản phẩm cá nhân hóa

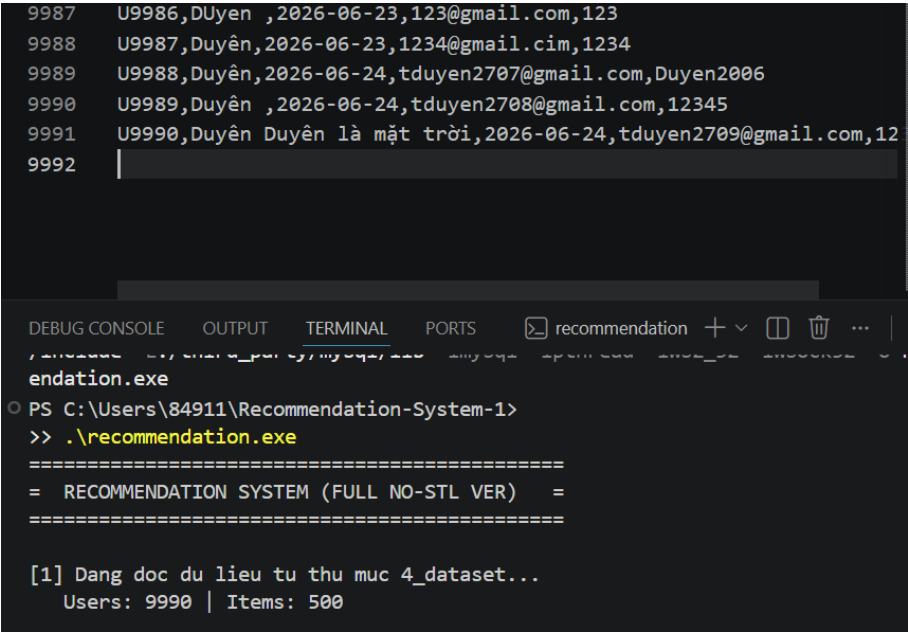
5.4 Kiểm thử các chức năng theo luồng nghiệp vụ

Hệ thống được kiểm thử theo luồng nghiệp vụ từ đầu đến cuối (end-to-end), bao gồm 37 trường hợp kiểm thử (TC01–TC37) phân theo 10 nhóm chức năng.

5.4.1 Nhóm 1: Nạp và xử lý dữ liệu (TC01–TC03)

TC ID	Tên kiểm thử	Đầu vào	Kết quả mong đợi	Trạng thái
TC01	Nạp file users.csv	File 9990 dòng (1 header + 9991 data)	users.size() == 10000; không lỗi parse	Pass
TC02	Nạp file items.csv	File 501 dòng (1 header + 500 data)	items.size() == 500; trường price parse đúng kiểu double	Pass
TC03	Nạp file interactions.csv	File 100.001 dòng; rating có thể NULL	interactions.size() == 100000; NULL rating được parse thành 0.0	Pass

Bảng 5.2: Kiểm thử Nhóm 1: Nạp và xử lý dữ liệu



```
9987 U9986,Duyen ,2026-06-23,123@gmail.com,123
9988 U9987,Duyen,2026-06-23,1234@gmail.cim,1234
9989 U9988,Duyen,2026-06-24,tduyen2707@gmail.com,Duyen2006
9990 U9989,Duyen ,2026-06-24,tduyen2708@gmail.com,12345
9991 U9990,Duyen Duyen là mặt trời,2026-06-24,tduyen2709@gmail.com,12
9992 |
=====
= RECOMMENDATION SYSTEM (FULL NO-STL VER) =
=====
[1] Dang doc du lieu tu thu muc 4_dataset...
Users: 9990 | Items: 500
```

Hình 5.3: Kết quả kiểm thử TC01–TC03: Nạp dữ liệu từ CSV

5.4.2 Nhóm 2: Tính điểm tương tác -computeScore() (TC04–TC06)

TC ID	Tên kiểm thử	Đầu vào	Kết quả mong đợi	Trạng thái
TC04	Tính điểm hành vi đầy đủ	click=3, add_cart=1, purchase=1, rating=5.0	$3 \times 1 + 1 \times 2 + 1 \times 5 + 5 \times 3 = 25.0$	Pass
TC05	Tính điểm chỉ click	click=5, add_cart=0, purchase=0, rating=0.0	$5 \times 1 = 5.0$	Pass
TC06	Tính điểm tất cả bằng 0	click=0, add_cart=0, purchase=0, rating=0.0	0.0 - đây là Interaction không hợp lệ, không nên tồn tại trong CSV	Pass

Bảng 5.3: Kiểm thử Nhóm 2: Hàm tính điểm computeScore()

5.4.3 Nhóm 3: Xây dựng Rating Matrix (TC07–TC09)

TC ID	Tên kiểm thử	Đầu vào	Kết quả mong đợi	Trạng thái
TC07	Xây dựng ma trận	100.000 interactions	<code>matrix[uid][iid] > 0</code> cho mọi cặp đã tương tác	Pass
TC08	Truy vấn điểm tồn tại	uid="U00001", iid có trong tương tác	Trả về giá trị > 0	Pass
TC09	Truy vấn điểm không tồn tại	uid="U00001", iid không có trong tương tác	Trả về 0.0; không gây exception	Pass

Bảng 5.4: Kiểm thử Nhóm 3: Xây dựng Rating Matrix

5.4.4 Nhóm 4: Tính Similarity Matrix (TC10–TC12)

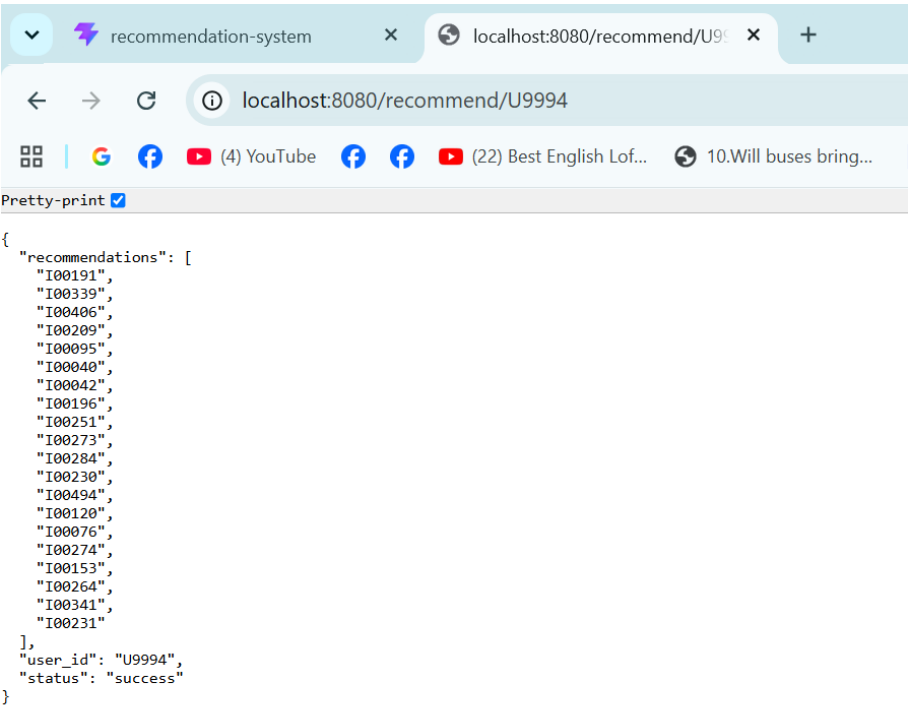
TC ID	Tên kiểm thử	Đầu vào	Kết quả mong đợi	Trạng thái
TC10	Xây dựng Similarity Matrix	RatingMatrix đã xây dựng	<code>simMatrix</code> có dữ liệu; tính đối xứng: <code>sim(A,B) == sim(B,A)</code>	Pass
TC11	Giá trị Cosine hợp lệ	Hai item có user chung	Giá trị $\in [0.0, 1.0]$	Pass
TC12	Cosine hai item không có user chung	Hai item tách biệt hoàn toàn	Trả về 0.0; không lưu vào <code>simMatrix</code>	Pass

Bảng 5.5: Kiểm thử Nhóm 4: Tính Similarity Matrix

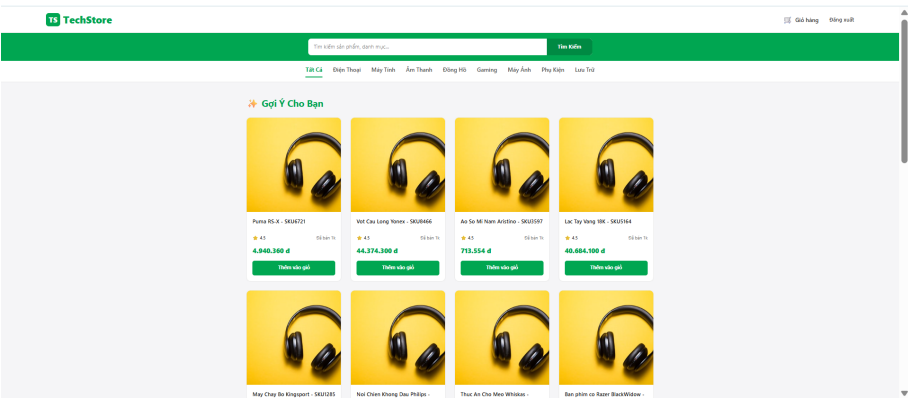
5.4.5 Nhóm 5: Engine gợi ý cá nhân hóa - `forUser()` (TC13–TC16)

TC ID	Tên kiểm thử	Đầu vào	Kết quả mong đợi	Trạng thái
TC13	Gợi ý cho user hợp lệ	uid="U00001", topN=10	Danh sách ≤ 10 item; không có item user đã tương tác	Pass
TC14	Kết quả sắp xếp giảm dần	uid hợp lệ, topN=5	<code>score[0] $\geq$ score[1] $\geq \dots \geq$ score[4]</code>	Pass
TC15	Không trùng item đã tương tác	uid="U00001"	Không có item nào trong seen set xuất hiện trong kết quả	Pass
TC16	Gợi ý khi user tương tác ít	uid chỉ có 1 interaction	Vẫn trả về kết quả hợp lệ, có thể ít hơn topN	Pass

Bảng 5.6: Kiểm thử Nhóm 5: Engine gợi ý cá nhân hóa



Hình 5.4: Kết quả kiểm thử TC13–TC16: Gợi ý sản phẩm cá nhân hóa



Hình 5.5: Kết quả kiểm thử TC13–TC16: Gợi ý sản phẩm cá nhân hóa

5.4.6 Nhóm 6: Gợi ý Cold Start (TC17–TC18)

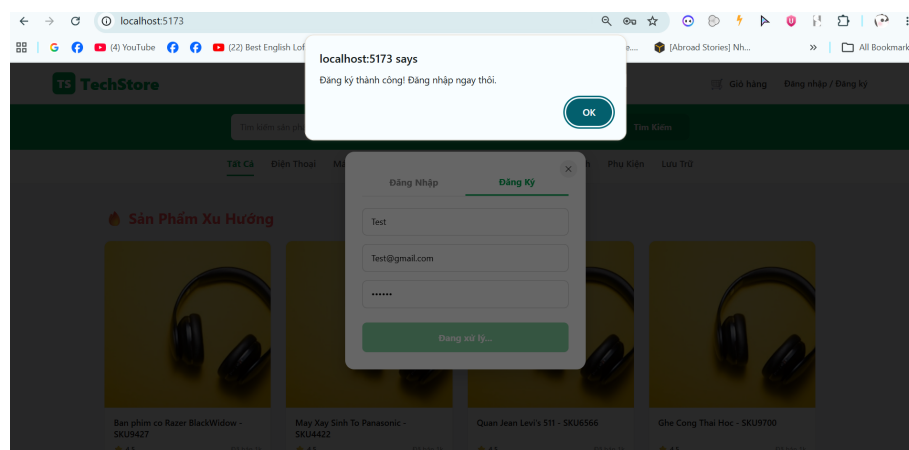
TC ID	Tên kiểm thử	Đầu vào	Kết quả mong đợi	Trạng thái
TC17	User mới - Cold Start	uid=“U99999” (không tồn tại trong matrix)	Log “la nguoi dung moi”; kết quả là Top-N sản phẩm phổ biến nhất, không rỗng	Pass
TC18	Cold Start sắp xếp đúng	uid mới, topN=5	itemTotalScore giảm dần; item[0] có tổng điểm cao nhất toàn hệ thống	Pass

Bảng 5.7: Kiểm thử Nhóm 6: Gợi ý Cold Start

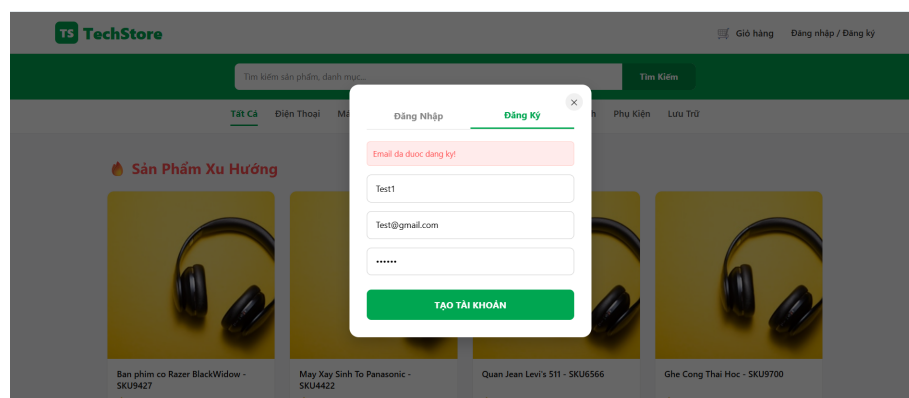
5.4.7 Nhóm 7: Xác thực người dùng - Auth (TC19–TC23)

TC ID	Tên kiểm thử	Đầu vào	Kết quả mong đợi	Trạng thái
TC19	Đăng ký thành công	name="Test", email="test@mail.com", pwd="123456"	status="success"; trả về user_id mới; ghi vào users.csv	Pass
TC20	Đăng ký email trùng	email đã tồn tại trong hệ thống	status="error"; message="Email đã được đăng ký!"	Pass
TC21	Đăng nhập đúng thông tin	email và password khớp	status="success"; trả về token và user_id	Pass
TC22	Đăng nhập sai mật khẩu	email đúng, password sai	status="error"; message="Sai mật khẩu!"	Pass
TC23	Đăng xuất	user_id hợp lệ đang đăng nhập	Session bị xóa; logged_in_user_id = ""	Pass

Bảng 5.8: Kiểm thử Nhóm 7: Xác thực người dùng



Hình 5.6: Kết quả kiểm thử TC19–TC23: Luồng xác thực người dùng

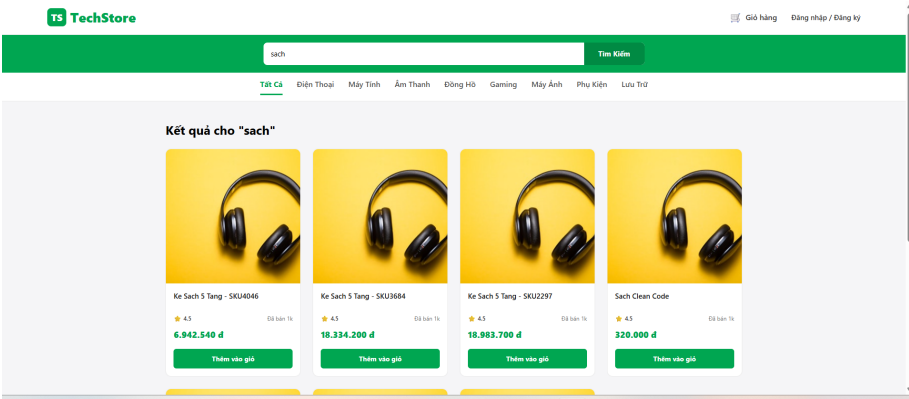


Hình 5.7: Kết quả kiểm thử TC19–TC23: Luồng xác thực người dùng

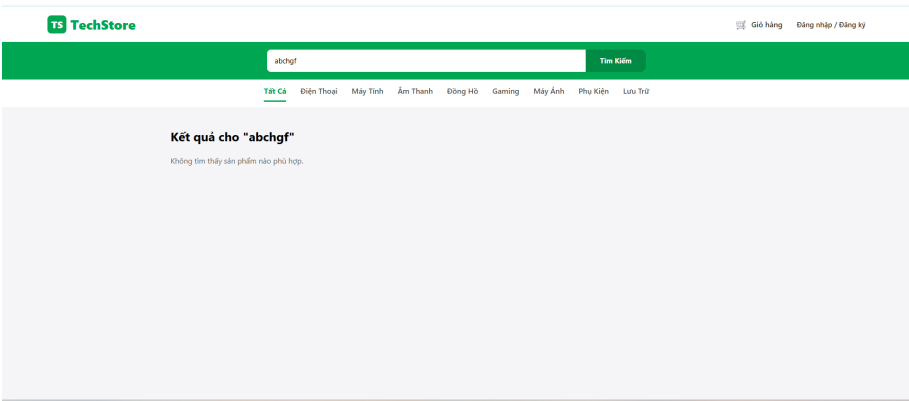
5.4.8 Nhóm 8: Quản lý sản phẩm (TC24–TC27)

TC ID	Tên kiểm thử	Đầu vào	Kết quả mong đợi	Trạng thái
TC24	Tìm kiếm sản phẩm đúng từ khóa	keyword="sach"	Trả về danh sách item có tên chứa "sach" (case-insensitive)	Pass
TC25	Tìm kiếm từ khóa không tồn tại	keyword="xyznotexist"	Trả về danh sách rỗng, không lỗi	Pass
TC26	Lọc theo danh mục và sắp xếp giá	category="Books", sortBy="price_asc"	Chỉ trả về sản phẩm category=Books; sắp xếp giá tăng dần	Pass
TC27	Xem chi tiết sản phẩm tồn tại	item_id="I001"	Trả về đúng thông tin Item I001 (Laptop Dell XPS 15)	Pass

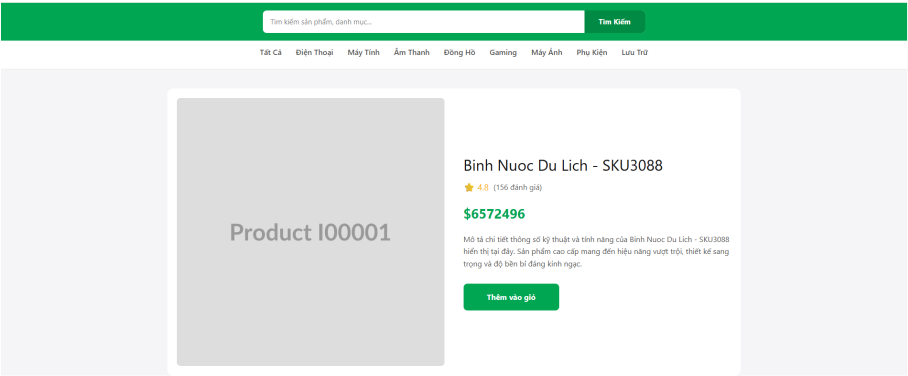
Bảng 5.9: Kiểm thử Nhóm 8: Quản lý sản phẩm



Hình 5.8: Kết quả kiểm thử TC24–TC27: Quản lý sản phẩm



Hình 5.9: Kết quả kiểm thử TC24–TC27: Quản lý sản phẩm

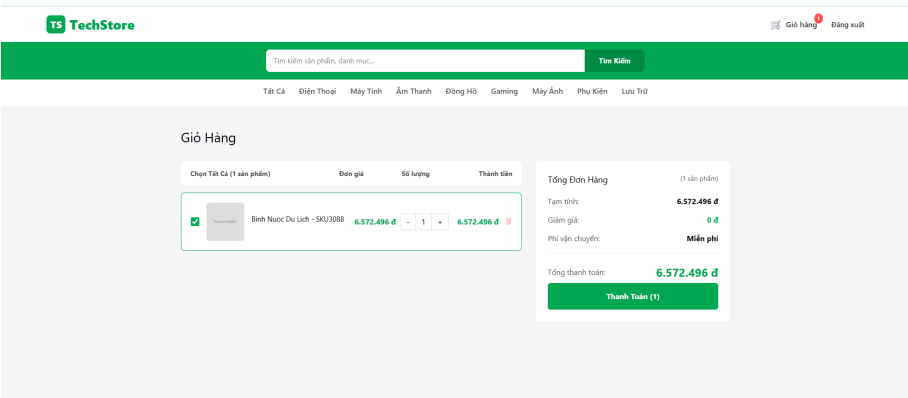


Hình 5.10: Kết quả kiểm thử TC24–TC27: Quản lý sản phẩm

5.4.9 Nhóm 9: Quản lý giỏ hàng (TC28–TC31)

TC ID	Tên kiểm thử	Đầu vào	Kết quả mong đợi	Trạng thái
TC28	Thêm sản phẩm mới vào giỏ	uid=“U001”, iid=“I001”, qty=1	CartItem được tạo; subtotal=price*qty	Pass
TC29	Thêm sản phẩm đã có trong giỏ	uid=“U001”, iid=“I001”, qty=2 (đã có qty=1)	quantity cộng dồn thành 3; subtotal cập nhật tương ứng	Pass
TC30	Cập nhật số lượng	uid=“U001”, iid=“I001”, newQty=0	Item bị xóa khỏi giỏ; tổng giỏ hàng giảm tương ứng	Pass
TC31	Xóa sản phẩm khỏi giỏ	uid=“U001”, iid=“I013”	Item “I013” bị loại; giỏ hàng chỉ còn các item còn lại	Pass

Bảng 5.10: Kiểm thử Nhóm 9: Quản lý giỏ hàng

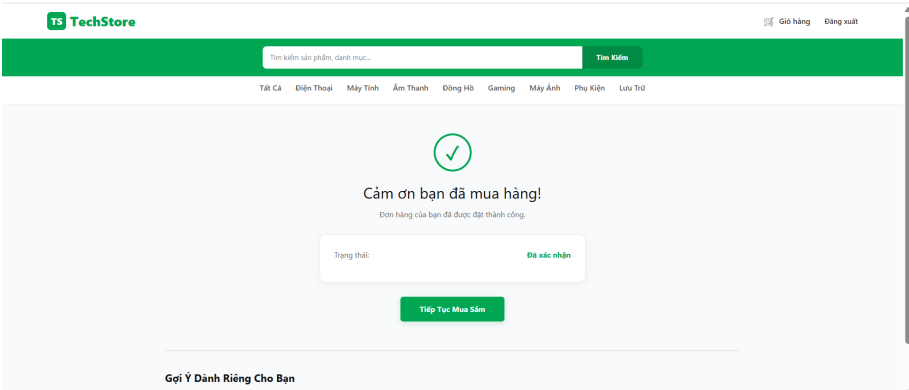


Hình 5.11: Kết quả kiểm thử TC28–TC31: Quản lý giỏ hàng

5.4.10 Nhóm 10: Đặt hàng và gợi ý chéo (TC32–TC37)

TC ID	Tên kiểm thử	Đầu vào	Kết quả mong đợi	Trạng thái
TC32	Tạo đơn hàng từ giỏ hợp lệ	uid="U001"; giỏ có 3 sản phẩm	status="success"; trả về order_id; giỏ hàng bị xóa	Pass
TC33	Tạo đơn từ giỏ rỗng	uid="U001"; giỏ rỗng	status="error"; message="Gio hang trong!"	Pass
TC34	Xem chi tiết đơn hàng	order_id hợp lệ vừa tạo	Trả về đủ thông tin: order_id, user_id, items, total, status="Pending"	Pass
TC35	Gợi ý chéo sau mua	order_id chứa sản phẩm category="Books"	Trả về ≤ 5 sản phẩm cùng category Books nhưng không trùng với items trong đơn	Pass
TC36	API /recommend/<uid> trả về JSON đúng	GET /recommend/U00001	HTTP 200; JSON có field recommendations là array các item_id	Pass
TC37	API /cart/get phản hồi đúng trạng thái giỏ	POST /cart/get với {user_id: "U001"}	HTTP 200; JSON có items và total khớp với state giỏ hàng hiện tại	Pass

Bảng 5.11: Kiểm thử Nhóm 10: Đặt hàng, gợi ý chéo và API



Hình 5.12: Kết quả kiểm thử TC32–TC37: Đặt hàng, gợi ý chéo và kiểm thử API

5.5 Tổng hợp kết quả kiểm thử

Nhóm kiểm thử	Số TC	PASS	FAIL
Nhóm 1: Nạp dữ liệu	3	x	
Nhóm 2: Tính điểm <code>computeScore()</code>	3	x	
Nhóm 3: Rating Matrix	3	x	
Nhóm 4: Similarity Matrix	3	x	
Nhóm 5: Gợi ý cá nhân hóa	4	x	
Nhóm 6: Cold Start	2	x	
Nhóm 7: Xác thực (Auth)	5	x	
Nhóm 8: Quản lý sản phẩm	4	x	
Nhóm 9: Giỏ hàng	4	x	
Nhóm 10: Đặt hàng và API	6	x	
Tổng cộng	37		

Bảng 5.12: Bảng tổng hợp kết quả kiểm thử 37 test case

Chương 6

Kết luận và Hướng phát triển

6.1 Kết quả đạt được

Sau quá trình triển khai, nhóm đã hoàn thành các mục tiêu đề ra ở Chương 1, cụ thể:

- Xây dựng thành công một hệ thống có kiến trúc bốn tầng rõ ràng, kết nối liền mạch giữa giao diện Frontend (ReactJS + Vite) và lõi xử lý thuật toán C++ thông qua REST API (Crow + ASIO), với cơ chế CORS middleware xử lý đầy đủ cả request OPTIONS lẫn các phương thức GET/POST.
- Làm chủ hoàn toàn các cấu trúc dữ liệu nền tảng bằng cách tự cài đặt `MyVector`, `MyMap`, `MyPair` thông qua con trỏ và quản lý bộ nhớ thủ công (`new[]/delete[]`), tuân thủ đầy đủ “Ba quy tắc lớn” (Destructor, Copy Constructor, Copy Assignment Operator) để tránh rò rỉ bộ nhớ - không phụ thuộc vào bất kỳ container nào của STL.
- Cài đặt thành công thuật toán Item-based Collaborative Filtering kết hợp Cosine Similarity, có tối ưu hoá bằng kỹ thuật `smallerMap` giúp giảm độ phức tạp tính tương đồng từ $O(N_u)$ xuống $O(d)$ với d là mật độ tương tác trung bình - một cải tiến có ý nghĩa thực tiễn rõ rệt trên ma trận thưa.
- Giải quyết bài toán Cold Start cho người dùng mới bằng chiến lược fallback phi cá nhân hoá (Popularity-based), đảm bảo hệ thống không bao giờ trả về danh sách gợi ý rỗng.
- Xây dựng đầy đủ các dịch vụ nghiệp vụ (`UserManager`, `ItemManager`, `InteractionManager`) phục vụ toàn bộ vòng đời tương tác: đăng ký/dăng nhập, tìm kiếm/lọc sản phẩm, giỏ hàng, đặt hàng và gợi ý chéo (cross-sell) sau khi mua.
- Tiến hành kiểm thử có hệ thống qua 37 trường hợp kiểm thử (TC01–TC37, Chương 5), bao trùm từ việc nạp dữ liệu, tính điểm tương tác, gợi ý cá nhân hoá, Cold Start, đến các API nghiệp vụ.

6.2 Tổng kết các kỹ thuật lập trình đã vận dụng

Nhóm kỹ thuật	Nội dung cụ thể
Quản lý bộ nhớ động	<code>new[]/delete[]</code> , chiến lược nhân đôi sức chứa (doubling strategy) trong <code>MyVector::reserveMore()</code> , Deep Copy trong Copy Constructor/Copy Assignment của <code>MyVector</code> và <code>MyMap</code> .
Lập trình hướng đối tượng	Đóng gói (Encapsulation) qua getter/setter trong <code>User/Item/Interaction</code> ; tách lớp theo nguyên lý Single Responsibility; phân tầng Model-Service (<code>1_models</code> ↔ <code>6_services</code>).
Cấu trúc dữ liệu tự cài đặt	Bảng băm với Separate Chaining (<code>MyMap</code>), mảng động khẩu hao $O(1)$ (<code>MyVector</code>), cặp khóa-giá trị template (<code>MyPair</code>).
Giải thuật	Bubble Sort $O(n^2)$ để xếp hạng Top-N gợi ý; tối ưu Cosine Similarity bằng kỹ thuật duyệt tập nhỏ hơn (<code>smallerMap</code>).
Lập trình hệ thống & mạng	HTTP server bất đồng bộ với Crow/ASIO, đa luồng (<code>std::thread</code> cho server song song với demo console), xử lý JSON (<code>crow::json::wvalue</code>), CORS middleware.
Xử lý chuỗi & I/O	Tokenization CSV bằng <code>std::stringstream</code> , <code>trim()</code> loại bỏ khoảng trắng/CRLF, <code>escapeField()</code> theo chuẩn CSV khi ghi tệp.
Tích hợp hệ thống ngoài	Kết nối MySQL qua MySQL Connector/C (<code>mysql_real_connect</code> , <code>mysql_real_escape_string</code> chống SQL Injection).

Bảng 6.1: Tổng kết các kỹ thuật lập trình đã vận dụng

6.3 Hạn chế của hệ thống

Hạn chế	Mô tả
Lưu trữ bằng CSV	Hiệu năng ghi giảm khi tập dữ liệu phình to lên hàng triệu dòng; không hỗ trợ giao dịch (transaction) hay truy cập đồng thời an toàn (concurrent write) như một RDBMS thực thụ.
Sparsity (loãng dữ liệu)	Thuật toán Item-based CF có thể cho kết quả kém chính xác khi ma trận tương tác quá thưa (nhiều người dùng/sản phẩm có rất ít tương tác).
Độ phức tạp $O(n^2)$ của <code>SimilarityMatrix::build()</code>	Dù đã tối ưu phần tính Cosine bằng <code>smallerMap</code> , việc duyệt mọi cặp sản phẩm vẫn có chi phí toàn cục bậc hai theo số lượng sản phẩm, gây khó khăn khi mở rộng lên hàng trăm nghìn sản phẩm.
Bubble Sort $O(n^2)$	Việc xếp hạng kết quả gợi ý dùng Bubble Sort thay vì các thuật toán $O(n \log n)$ (Quick Sort, Merge Sort), phù hợp với quy mô Top-N nhỏ hiện tại nhưng sẽ là điểm nghẽn nếu danh sách ứng viên lớn hơn đáng kể.
Bảo mật mật khẩu	Mật khẩu người dùng hiện được lưu dạng plaintext (<code>User::password</code>) chưa qua băm (hashing) như bcrypt/Argon2, chưa phù hợp triển khai thực tế.
Cold Start sản phẩm mới	Hệ thống mới xử lý Cold Start cho người dùng mới; sản phẩm mới chưa có tương tác vẫn chưa có cơ chế gợi ý chuyên biệt.
Không xử lý ngữ cảnh	Hệ thống chưa tính đến các yếu tố thời gian, vị trí địa lý hay thiết bị truy cập khi sinh gợi ý.

Bảng 6.2: Các hạn chế hiện tại của hệ thống

6.4 Hướng phát triển và nâng cấp tương lai

Ngắn hạn

- Băm mật khẩu (bcrypt/Argon2) thay cho lưu plaintext; bổ sung JWT cho cơ chế xác thực token thay vì token sinh thủ công bằng `time()` + số ngẫu nhiên.
- Thay Bubble Sort bằng Quick Sort/`std::sort`-tương đương tự cài đặt khi danh sách ứng viên gợi ý mở rộng.
- Cache kết quả `SimilarityMatrix` ra tệp nhị phân, định kỳ tính lại theo lịch (batch job) thay vì tính lại toàn bộ mỗi lần khởi động server.

Trung hạn

- Chuyển nguồn dữ liệu chính từ CSV sang MySQL/PostgreSQL ở quy mô sản xuất, tận dụng `DatabaseManager` đã có sẵn, bổ sung transaction và index.
- Bổ sung Content-Based Filtering (dựa trên category, mô tả sản phẩm) kết hợp với Item-based CF hiện tại theo mô hình Hybrid Recommendation, đồng thời xử lý Cold Start cho sản phẩm mới.
- Cập nhật gợi ý theo thời gian thực (incremental update) khi có tương tác mới, thay vì chỉ xây dựng `SimilarityMatrix` một lần lúc khởi động.

Dài hạn

- Áp dụng các mô hình Matrix Factorization (SVD, ALS) hoặc Neural Collaborative Filtering để cải thiện độ chính xác trên dữ liệu thưa.
- Tính đến yếu tố ngữ cảnh (Context-Aware Recommendation: thời gian, thiết bị, vị trí) và đảm bảo tính công bằng, đa dạng (Fairness & Diversity), tránh hiện tượng bóng lọc thông tin (filter bubble).
- Mở rộng kiến trúc theo hướng phân tán (ví dụ tách `SimilarityMatrix` thành dịch vụ tính toán độc lập) để đáp ứng quy mô dữ liệu lớn hơn nhiều lần so với phạm vi thử nghiệm hiện tại.

Tài liệu tham khảo

- [1] Forrester Research, “Global retail e-commerce forecast 2024–2028,” 2024.
- [2] Gartner, “Information overload and customer abandonment in e-commerce,” 2023.
- [3] McKinsey & Company, “The next frontier of personalized marketing,” 2025.
- [4] Salesforce Research, “Personalization, data security, and speed drive customer loyalty,” 2023.
- [5] M. D. Ekstrand, J. T. Riedl, and J. A. Konstan, “Collaborative filtering recommender systems,” *Foundations and Trends in Human–Computer Interaction*, 2011.
- [6] G. Linden, B. Smith, and J. York, “Amazon.com recommendations: Item-to-item collaborative filtering,” *IEEE Internet Computing*, vol. 17, no. 1, pp. 76–80, 2003.
- [7] B. Sarwar, G. Karypis, J. Konstan, and J. Riedl, “Item-based collaborative filtering recommendation algorithms,” in *Proceedings of the 10th International Conference on World Wide Web (WWW '01)*, 2001.
- [8] F. Ricci, L. Rokach, and B. Shapira, *Introduction to Recommender Systems Handbook*. Springer US, 2011.
- [9] A. I. Schein, A. Popescul, L. H. Ungar, and D. M. Pennock, “Methods and metrics for cold-start recommendations,” in *Proceedings of the 25th Annual International ACM SIGIR Conference*, 2002.
- [10] B. Stroustrup and H. Sutter, “C++ core guidelines,” 2024.
- [11] M. A. Weiss, *Data Structures and Algorithm Analysis in C++*. Pearson Education, 4th ed., 2013.
- [12] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*. MIT Press, 3rd ed., 2009.
- [13] N. Đức Nghĩa and T. Q. Tùng, *Giáo trình Cấu trúc dữ liệu và giải thuật*. Nhà xuất bản Bách Khoa Hà Nội, 2013.

Phụ lục A: Hướng dẫn triển khai hệ thống

Phụ lục này mô tả quy trình thiết lập môi trường, biên dịch và khởi chạy hệ thống từ đầu trên Windows.

A.1 Yêu cầu hệ thống

Thành phần	Phiên bản tối thiểu	Ghi chú
Hệ điều hành	Windows 10/11	x64
Trình biên dịch	MinGW-w64 (g++ 8.1+)	Hỗ trợ C++17
MySQL Server	8.0+	Chạy local trên 127.0.0.1:3306
Git	Bất kỳ	Clone repo
CMake	Không bắt buộc	Chỉ dùng nếu muốn build qua CMake

Bảng 3: Yêu cầu hệ thống

Lưu ý quan trọng: Hệ thống sử dụng **CrowCpp** làm HTTP server và **MySQL Connector/C** để kết nối MySQL. Cả hai thư viện đã được đóng gói sẵn trong thư mục `include/` và `third_party/mysql/`. Bạn không cần cài thêm thư viện bên ngoài.

A.2 Quy trình triển khai (8 bước)

Bước 1 - Clone repository

```
1 git clone https://github.com/ndtchumoa/Recommendation-System.git
2 cd Recommendation-System
```

Listing 1: Clone repository

Bước 2 - Kiểm tra cấu trúc thư mục

Đảm bảo cây thư mục đầy đủ:

```
Recommendation-System/
├── 1_models/ ..... Entity classes
├── 2_data/ ..... CSV Loader & DatabaseManager
│   ├── db_config.example.h ..... Mẫu cấu hình DB
├── 3_core/ ..... Core algorithms
├── 4_dataset/ ..... CSV dữ liệu mẫu
├── 6_services/ ..... Service layer
├── include/ ..... CrowCpp + ASIO headers
├── third_party/mysql/ ..... MySQL Connector/C headers + libs
│   ├── include/
│   └── lib/
```



Bước 3 - Cài đặt MySQL Server (nếu chưa có)

Cách A - XAMPP (khuyến dùng cho dev):

1. Tải [XAMPP](#) và cài đặt.
2. Mở XAMPP Control Panel → Start **MySQL** (port 3306, user: root, password: rỗng "").

Cách B - MySQL Installer (bản đầy đủ):

1. Tải [MySQL Installer](#) và cài MySQL Server 8.0+.
2. Ghi nhớ **port**, **user** và **password** bạn đã đặt.

Bước 4 - Tạo database

Mở MySQL client (Command Prompt / Workbench) và chạy:

```

1 CREATE DATABASE IF NOT EXISTS recommendation_db
2     CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
3 -- (tuy chọn) Tao user riêng cho ứng dụng
4 CREATE USER 'rec_app'@'localhost' IDENTIFIED BY 'your_password';
5 GRANT ALL PRIVILEGES ON recommendation_db.* TO 'rec_app'@'localhost';
6 FLUSH PRIVILEGES;
  
```

Listing 2: Tạo database

Ứng dụng sẽ tự gọi `createTables()` khi khởi chạy để tạo các bảng `users`, `items`, `interactions`
 - bạn không cần tạo bảng bằng tay.

Bước 5 - Cấu hình kết nối database

Mỗi thành viên trong nhóm cần tạo file cấu hình DB **cá nhân** (không commit lên git):

```

1 # PowerShell
2 Copy-Item 2_data\db_config.example.h 2_data\db_config.h
  
```

Listing 3: Sao chép file cấu hình DB

Mở file `2_data/db_config.h` và sửa các thông số:

```

1 static const std::string DB_HOST = "127.0.0.1";
2 static const std::string DB_USER = "root";           // Ten dang nhap
3 static const std::string DB_PASS = "";
4 // Mat khau (XAMPP: "")
5 static const std::string DB_NAME = "recommendation_db";
6 static const unsigned int DB_PORT = 3306;
  
```

Listing 4: Nội dung `db_config.h` cần chỉnh sửa

File `db_config.h` đã được liệt kê trong `.gitignore` → **an toàn, không bị đẩy lên GitHub.**

Bước 6 - Kiểm tra MySQL Connector/C

Thư mục `third_party/mysql/` đã được commit kèm trong repo, bao gồm:

- `include/` - chứa `mysql.h` và các header MySQL
- `lib/libmysql.dll` - thư viện động (runtime)
- `lib/libmysql.a` - import library cho MinGW g++ (đã được tạo từ `libmysql.dll`)

Nếu thư mục này còn đầy đủ → **bỏ qua bước này**. Nếu thiếu, làm theo hướng dẫn chi tiết tại `SETUP_MYSQL_CONNECTOR.md`.

Bước 7 - Biên dịch (build)

Cách A - Build bằng VSCode Task (Ctrl+Shift+B):

1. Mở project trong VSCode.
2. Nhấn Ctrl+Shift+B → chọn task **build-recommendation**.
3. Task sẽ tự động copy libmysql.dll ra thư mục gốc sau khi build.

Cách B - Build thủ công (Windows cmd):

```

1 g++ -std=c++17 -O2 -Wall ^
2 -Ithird_party/mysql/include ^
3 main.cpp 1_models/*.cpp 2_data/*.cpp 3_core/*.cpp ^
4 6_services/*.cpp ^
5 -Lthird_party/mysql/lib -llibmysql ^
6 -lws2_32 -lws2_32 -lws2_32 -lpthread ^
7 -o recommendation.exe
8
9 copy third_party\mysql\lib\libmysql.dll .

```

Listing 5: Lệnh build thủ công

Lưu ý: libmysql.dll phải nằm cùng thư mục với recommendation.exe khi chạy.

Bước 8 - Đưa dữ liệu mẫu vào MySQL (seed)

Cách A - Import tự động (khuyến dùng):

Chạy chương trình; main.cpp sẽ tự động đọc CSV và đồng bộ vào MySQL:

```
1 recommendation.exe
```

Cách B - Import từ file .sql:

```

1 mysql -u root -p recommendation_db < recommendationsystem_users.sql
2 mysql -u root -p recommendation_db < recommendationsystem_items.sql
3 mysql -u root -p recommendation_db <
  recommendationsystem_interactions.sql

```

Listing 6: Import dữ liệu mẫu từ SQL dump

(Thứ tự quan trọng: **users** → **items** → **interactions** vì có ràng buộc FOREIGN KEY.)

A.3 Khởi chạy và kiểm tra hệ thống

Sau khi build thành công, chạy:

```
1 recommendation.exe
```

Chương trình sẽ: (1) nạp dữ liệu từ 4_dataset/*.csv, (2) đồng bộ vào MySQL, (3) khởi tạo Recommender Engine, (4) chạy demo console, (5) khởi động API server tại <http://localhost:8080>. Nhấn Ctrl+C để tắt server.

Kiểm tra API bằng trình duyệt hoặc curl:

```

1 # Lay danh sach san pham
2 curl http://localhost:8080/products
3
4 # Lay goi y cho user U001
5 curl http://localhost:8080/recommend/U001
6
7 # Lay trending items
8 curl http://localhost:8080/trending

```

Listing 7: Ví dụ gọi API

A.4 Danh sách các API Endpoints

Method	Endpoint	Chức năng	Body (JSON)
GET	/products	Lấy danh sách sản phẩm	-
GET	/trending	Lấy top 20 sản phẩm thịnh hành	-
GET	/recommend/<user_id>	Gợi ý sản phẩm cho user	-
GET	/product/<item_id>	Chi tiết sản phẩm	-
POST	/register	Đăng ký tài khoản	{name, email, password}
POST	/login	Đăng nhập	{email, password}
POST	/logout	Đăng xuất	{user_id}
POST	/cart/add	Thêm vào giỏ hàng	{user_id, item_id, quantity}
POST	/cart/remove	Xóa khỏi giỏ hàng	{user_id, item_id}
POST	/cart/get	Xem giỏ hàng	{user_id}
POST	/track	Ghi nhận tương tác	{user_id, item_id, action}
POST	/order/create	Tạo đơn hàng	{user_id}

Bảng 4: Danh sách các API Endpoints

Tất cả endpoint đều hỗ trợ CORS (có xử lý OPTIONS preflight).

Phụ lục B: Mã nguồn các thuật toán cốt lõi

Phụ lục này trình bày mã nguồn hoàn chỉnh của năm thuật toán/hàm cốt lõi nhất trong hệ thống: hàm tính điểm tương tác, xây dựng ma trận đánh giá, tính độ tương đồng Cosine, gợi ý cá nhân hóa và gợi ý Cold Start.

B.1 Hàm `Interaction::computeScore()` - Tính điểm tương tác tổng hợp

File: `1_models/Interaction.cpp`

Đây là hàm nền tảng của toàn hệ thống. Thay vì chỉ dựa vào rating số sao, hàm kết hợp bốn tín hiệu hành vi với trọng số phản ánh mức độ cam kết thực sự: mua hàng (5.0) > đánh giá (3.0) > thêm giỏ (2.0) > click (1.0).

```

1  // Cac trong so hanh vi (hang so static)
2  const double Interaction::WEIGHT_CLICK      = 1.0;
3  const double Interaction::WEIGHT_ADD_CART  = 2.0;
4  const double Interaction::WEIGHT_PURCHASE  = 5.0;
5  const double Interaction::WEIGHT_RATING    = 3.0;
6
7  // Cong thuc tinh diem co trong so:
8  // Score = (Click * W_click) + (Cart * W_cart)
9  //         + (Purchase * W_purchase) + (Rating * W_rating)
10 double Interaction::computeScore() const {
11     double score = (click_count      * WEIGHT_CLICK)
12                   + (add_cart_count  * WEIGHT_ADD_CART)
13                   + (purchase_count  * WEIGHT_PURCHASE)
14                   + (rating          * WEIGHT_RATING);
15     return score;
16 }
```

Listing 8: Hàm `computeScore()` - File `1_models/Interaction.cpp`

B.2 Hàm `RatingMatrix::buildMatrix()` - Xây dựng ma trận User-Item

File: `3_core/RatingMatrix.cpp`

Hàm duyệt qua toàn bộ `MyVector<Interaction>` và xây dựng ma trận thưa hai chiều `matrix[user_id][item_id] = score`. Sử dụng `MyMap` lồng nhau để chỉ lưu các ô có giá trị khác 0, tiết kiệm bộ nhớ đáng kể so với ma trận đầy.

```

1 void RatingMatrix::buildMatrix(
2     const MyVector<Interaction>& interactions) {
3     for (int i = 0; i < interactions.size(); ++i) {
4         std::string u_id = interactions[i].getUserId();
5         std::string i_id = interactions[i].getItemId();
```

```

6
7      // Goi computeScore() tu lop Interaction
8      // de lay diem tong hop co trong so
9      double score = interactions[i].computeScore();
10
11      // Gan vao ma tran (tu dong tao moi neu chua ton tai)
12      matrix[u_id][i_id] = score;
13  }
14 }

```

Listing 9: Hàm buildMatrix() - File 3_core/RatingMatrix.cpp

Sau buildMatrix(), có thể gọi thêm normalizeMatrix() để thực hiện Mean-Centering - trừ điểm trung bình của mỗi user khỏi từng score, giúp chuẩn hóa xu hướng đánh giá cao/thấp cá nhân.

B.3 Hàm SimilarityMatrix::calculateCosine() - Tính độ tương đồng Cosine tối ưu

File: 3_core/SimilarityMatrix.cpp

Hàm tính Cosine Similarity giữa hai vector sản phẩm. Kỹ thuật cốt lõi là `smallerMap`: chỉ duyệt qua vector có ít phần tử hơn và dùng tra cứu $O(1)$ của bảng băm để kiểm tra tập giao. Điều này giảm độ phức tạp từ $O(N_u)$ xuống $O(d)$ với d là mật độ tương tác.

```

1  double SimilarityMatrix::calculateCosine(
2      const MyMap<std::string, double>& itemA_users,
3      const MyMap<std::string, double>& itemB_users) const {
4      double dotProduct = 0.0, normA = 0.0, normB = 0.0;
5
6      // 1. Tinh do dai vector A
7      MyVector<std::string> keysA = itemA_users.keys();
8      for (int i = 0; i < keysA.size(); ++i) {
9          double val;
10         itemA_users.find(keysA[i], val);
11         normA += val * val;
12     }
13
14     // 2. Tinh do dai vector B
15     MyVector<std::string> keysB = itemB_users.keys();
16     for (int i = 0; i < keysB.size(); ++i) {
17         double val;
18         itemB_users.find(keysB[i], val);
19         normB += val * val;
20     }
21
22     // Neu mot trong hai vector bang 0 -> cosine = 0
23     if (normA == 0.0 || normB == 0.0) return 0.0;
24
25     // 3. Ky thuat smallerMap: duyet qua map nho hon
26     // de giam do phuc tap tinh tich vo huong
27     const MyMap<std::string, double>* smallerMap = &itemA_users;
28     const MyMap<std::string, double>* largerMap = &itemB_users;
29     if (itemA_users.size() > itemB_users.size()) {
30         smallerMap = &itemB_users;
31         largerMap = &itemA_users;
32     }
33
34     MyVector<std::string> smallerKeys = smallerMap->keys();
35     for (int i = 0; i < smallerKeys.size(); ++i) {
36         double scoreA, scoreB;
37         smallerMap->find(smallerKeys[i], scoreA);

```

```

38         // Chỉ tính nếu User này cũng tương tác với item bên kia
39         if (largerMap->find(smallerKeys[i], scoreB)) {
40             dotProduct += scoreA * scoreB;
41         }
42     }
43
44     // 4. Tra về cosine
45     return dotProduct / (std::sqrt(normA) * std::sqrt(normB));
46 }

```

Listing 10: Hàm calculateCosine() tối ưu - File 3_core/SimilarityMatrix.cpp

B.4 Hàm Recommender::forUser() - Gợi ý cá nhân hóa

File: 3_core/Recommender.cpp

Hàm tổng hợp kết quả gợi ý cho một người dùng cụ thể. Logic cốt lõi: với mỗi item user đã tương tác, lấy danh sách hàng xóm từ SimilarityMatrix, tích lũy điểm dự đoán theo công thức weighted sum, rồi sắp xếp và cắt TopN.

```

1  RecommendResult Recommender::forUser(
2      const std::string& uid, int topN) const {
3      // User mới chưa có interaction -> Cold Start
4      if (rm_.isNewUser(uid)) {
5          std::cout << "[Recommender] '" << uid
6                  << "' là người dùng mới -> Cold Start\n";
7          return coldStart(topN);
8      }
9
10     const MyVector<std::string>& allItems = rm_.items();
11
12     // Tap item user đã tương tác (dùng MyMap như một set)
13     MyMap<std::string, bool> seen;
14     for (int i = 0; i < allItems.size(); ++i) {
15         if (rm_.get(uid, allItems[i]) > 0.0)
16             seen[allItems[i]] = true;
17     }
18
19     // Tích lũy điểm dự đoán cho các item chưa tương tác
20     // predicted[j] = Sum(sim(i,j) * score(u,i)) / Sum|sim(i,j)|
21     MyMap<std::string, double> num, den;
22     MyVector<std::string> seenKeys = seen.keys();
23     for (int s = 0; s < seenKeys.size(); ++s) {
24         std::string iid = seenKeys[s];
25         double userScore = rm_.get(uid, iid);
26         // Lấy danh sách hàng xóm từ SimilarityMatrix
27         MyVector<MyPair<std::string, double>> neighbors =
28             sm_.topK(iid, allItems.size());
29         for (int n = 0; n < neighbors.size(); ++n) {
30             std::string nbr = neighbors[n].first;
31             double sim = neighbors[n].second;
32             if (seen.contains(nbr)) continue; // Đã xem -> bỏ
33             num[nbr] += sim * userScore;
34             den[nbr] += std::abs(sim);
35         }
36     }
37
38     // Tính điểm cuối và gom vào result
39     RecommendResult result;
40     MyVector<std::string> numKeys = num.keys();
41     for (int i = 0; i < numKeys.size(); ++i) {

```



```

42     std::string iid = numKeys[i];
43     double d;
44     if (!den.find(iid, d) || d == 0.0) continue;
45     result.push_back(MyPair<std::string, double>(
46         iid, num[iid] / d));
47 }
48
49 // Sắp xếp giảm dần theo score (Bubble Sort)
50 for (int i = 0; i < result.size() - 1; ++i) {
51     for (int j = 0; j < result.size() - i - 1; ++j) {
52         if (result[j].second < result[j + 1].second) {
53             MyPair<std::string, double> tmp = result[j];
54             result[j] = result[j + 1];
55             result[j + 1] = tmp;
56         }
57     }
58 }
59
60 // Cat lấy TopN
61 if (result.size() > topN) {
62     RecommendResult truncated;
63     for (int i = 0; i < topN; ++i)
64         truncated.push_back(result[i]);
65     result = truncated;
66 }
67 return result;
68 }

```

Listing 11: Hàm forUser() - File 3_core/Recommender.cpp

B.5 Hàm Recommender::coldStart() - Gợi ý người dùng mới

File: 3_core/Recommender.cpp

Khi người dùng chưa có lịch sử tương tác, hàm này thay thế gợi ý cá nhân hóa bằng danh sách sản phẩm phổ biến nhất tính theo tổng điểm toàn hệ thống (itemTotalScore).

```

1  RecommendResult Recommender::coldStart(int topN) const {
2      MyVector<MyPair<std::string, double>> result;
3      MyVector<std::string> allItems = rm_.items();
4
5      // Tính tổng điểm của từng item trên toàn bộ hệ thống
6      for (int i = 0; i < allItems.size(); ++i) {
7          result.push_back(MyPair<std::string, double>(
8              allItems[i],
9              rm_.itemTotalScore(allItems[i])
10         ));
11     }
12
13     // Sắp xếp giảm dần theo tổng điểm (Bubble Sort)
14     for (int i = 0; i < result.size() - 1; ++i) {
15         for (int j = 0; j < result.size() - i - 1; ++j) {
16             if (result[j].second < result[j + 1].second) {
17                 MyPair<std::string, double> tmp = result[j];
18                 result[j] = result[j + 1];
19                 result[j + 1] = tmp;
20             }
21         }
22     }
23
24     // Cat lấy TopN sản phẩm phổ biến nhất

```

```
25     if (result.size() > topN) {  
26         MyVector<MyPair<std::string, double>> truncated;  
27         for (int i = 0; i < topN; ++i)  
28             truncated.push_back(result[i]);  
29         result = truncated;  
30     }  
31     return result;  
32 }
```

Listing 12: Hàm coldStart() - File 3_core/Recommender.cpp